Artificial Intelligence & Machine Learning

Unit I: Introduction to AI, Problem Solving and Search Strategies

Study Notes with Mind Maps, Flowcharts, Diagrams and Cheat Sheet

B.C.A. Full Stack · School of Computer & Systems Sciences · Jaipur National University, Jaipur · Course code BCAFSSE501T23 (3 credits, 100 marks)

Contents

1. Unit 1 at a Glance
2. What is Intelligence?
3. Foundations of Artificial Intelligence
4. History and Applications of AI
5. Intelligent Agents and Problem Formulation
6. Classic Problems: States, Operators and State Spaces
7. Search Strategies
8. Unit 1 Cheat Sheet
9. Important Questions
10. MCQ Practice
11. Glossary

Unit 1 at a Glance

This unit lays the groundwork for the whole course on Artificial Intelligence and Machine Learning (BCAFSSE501T23). It answers three big questions. First, what do we actually mean by "intelligence", and what does it mean for a machine to be intelligent? Second, where did the ideas behind AI come from, and how did the field develop from a 1956 summer workshop into today's large language models? Third, how can a computer program solve a problem it has never seen before by searching systematically through possibilities?

The third question takes up the largest share of the unit and of the exam paper. You will learn to **formulate** a problem precisely (initial state, operators, goal test, path cost), to draw its **state space**, and to apply the classic **uninformed search strategies** — breadth-first, uniform-cost, depth-first, depth-limited, iterative deepening and bidirectional search — with hand-worked traces. These ideas are reused in Unit 2 (informed and heuristic search) and in later topics such as planning and game playing.

Syllabus Coverage

Every phrase of the official syllabus is mapped below to the chapter where it is taught. Use this table as a checklist during revision.

Table 1.1: Syllabus phrases mapped to chapters

Syllabus phrase	Where it is covered
What is intelligence?	What is Intelligence? — types of intelligence, Turing test, Chinese room
Artificial intelligence (AI) — meaning	What is Intelligence? — definitions, four approaches, strong vs weak AI
Foundations of artificial intelligence	Foundations of Artificial Intelligence — eight contributing disciplines
History of AI	History and Applications of AI — timeline from 1943 to large language models
Problem solving	Intelligent Agents and Problem Formulation — problem-solving agents
Formulating problems	Intelligent Agents and Problem Formulation — five components
Problem types	Intelligent Agents and Problem Formulation — single-state, sensorless, contingency, exploration
States and operators	Intelligent Agents and Problem Formulation; Classic Problems worked in full
State space	Intelligent Agents and Problem Formulation; Classic Problems (state-space diagrams)
Search strategies	Search Strategies — measures, six uninformed strategies, comparison, bridge to Unit 2

Learning Outcomes

After studying this unit you should be able to:

- Define intelligence and artificial intelligence, and explain the Turing test, the total Turing test and Searle's Chinese room argument.

- Classify definitions of AI into the four approaches: thinking humanly, thinking rationally, acting humanly and acting rationally.
- Distinguish strong AI from weak AI, and narrow AI from general AI.
- Describe how philosophy, mathematics, economics, neuroscience, psychology, computer engineering, control theory and linguistics contributed to AI.
- Narrate the history of AI with correct dates for major milestones, including the two "AI winters".
- List real-world applications of AI across sectors.
- Describe an intelligent agent using PEAS and classify task environments.
- Formulate a problem as initial state, actions (operators), transition model, goal test and path cost.
- Identify problem types: single-state, sensorless (conformant), contingency and exploration; and toy versus real-world problems.
- Draw the state space of the water-jug, 8-puzzle, 8-queens, missionaries-and-cannibals, vacuum-world and route-finding problems.
- Trace BFS, UCS, DFS, DLS, IDS and bidirectional search on a given tree or graph, and state their completeness, optimality, time and space complexity.
- Explain generate-and-test and hill climbing, and say why informed search is needed.

How to Use These Notes

The notes are arranged in the order the syllabus is normally taught. Each teaching chapter follows the same pattern: explanation in short paragraphs, tables that compress the facts, diagrams you can reproduce in the exam, worked examples in shaded boxes, and a short summary at the end.

- **Key Point** boxes contain the single idea you must not miss.
- **Exam Tip** boxes tell you how examiners usually ask a topic and what earns full marks.
- **Memory Trick** boxes give mnemonics.
- **Worked Example** boxes solve a problem step by step — practise them with a pen before reading the solution.

For a 2-mark question, a crisp definition plus one example is enough. For a 5-mark question, add a diagram or table and two or three properties. For a 10-mark question, write an introduction, a labelled diagram, an algorithm or formulation, a worked trace, properties or a comparison table, and a conclusion. The back-matter (Cheat Sheet, Important Questions, MCQs and Glossary) is designed for the last two days before the exam.

> **Exam Tip:** In this unit, the most frequently asked long questions are: the water-jug problem with production rules, the comparison of uninformed search strategies, BFS versus DFS with a trace, the Turing test, and the foundations or history of AI. Make sure you can draw the related diagrams from memory.

Unit Mind Map

The mind map below shows the whole unit on one page. The eight branches correspond to the main topics; the leaves are the sub-topics most often asked in exams.



Figure 1.1: Unit 1 mind map — AI foundations and problem solving

Key Point: The unit moves from "what is AI" (concepts and history) to "how does AI solve problems" (formulation and search). The link between the two halves is the idea of a rational agent: an agent that chooses actions to achieve its goals, and search is one of the simplest ways of choosing those actions.

What is Intelligence?

Before we can build artificial intelligence, we need some idea of what intelligence itself is. This sounds like a philosophical question, and partly it is, but it has very practical consequences. The definition we choose decides what we try to build and how we judge whether we have succeeded.

Defining Intelligence

There is no single definition of **intelligence** accepted by all psychologists, philosophers and computer scientists. Most working definitions, however, share a common core: intelligence is the ability to learn from experience, to understand and reason about the world, to solve new problems, and to adapt behaviour to achieve goals in a changing environment.

A useful way to make the idea concrete is to list the **components of intelligence** that we observe in people and would like to see in machines:

- **Reasoning** — drawing conclusions from facts. Deductive reasoning goes from general rules to specific cases ("all men are mortal; Socrates is a man; so Socrates is mortal"). Inductive reasoning generalises from examples ("every swan I have seen is white, so swans are probably white"). Abductive reasoning finds the most likely explanation ("the grass is wet, so it probably rained").
- **Learning** — improving performance with experience: learning by being told, by examples, by trial and error (reinforcement) and by discovery.
- **Problem solving** — finding a sequence of actions that turns the current situation into a desired one, often by searching through alternatives.
- **Perception** — interpreting raw sensory input (images, sounds, touch) to build a model of the surroundings.
- **Linguistic intelligence** — understanding and producing natural language.
- **Planning and decision making** — choosing among alternatives under uncertainty, weighing costs and benefits.
- **Adaptation** — changing behaviour when the environment changes.

Types of Intelligence

Psychologist Howard Gardner proposed in 1983 that people have several relatively independent kinds of intelligence rather than one general ability. His **theory of multiple intelligences** is often quoted in AI introductions because it reminds us that "intelligent behaviour" has many forms. AI systems today are strong in some of these and very weak in others.

Table 1.2: Gardner's multiple intelligences and their AI analogues

Type of intelligence	Description	AI analogue
Linguistic	Using words, reading, writing, speaking	Natural language processing, chatbots
Logical-mathematical	Reasoning, calculation, logic	Theorem provers, solvers, expert systems
Spatial	Visualising and manipulating images and space	Computer vision, robot navigation
Musical	Recognising and producing rhythm, pitch, melody	Music generation and recognition

Type of intelligence	Description	AI analogue
Bodily-kinesthetic	Controlling body movement, handling objects	Robotics, motion control
Interpersonal	Understanding other people's intentions and moods	Sentiment analysis, social robots
Intrapersonal	Understanding oneself, self-reflection	Very limited in current AI
Naturalist	Recognising and classifying plants, animals, patterns in nature	Image classification of species

Human Intelligence versus Machine Intelligence

It is useful to compare natural (human) intelligence with artificial (machine) intelligence. The comparison is not "which is better" — each has different strengths.

Table 1.3: Human intelligence compared with machine intelligence

Aspect	Human intelligence	Machine (artificial) intelligence
Origin	Biological, evolved; develops through life	Engineered; designed and trained by people
Speed of calculation	Slow for arithmetic	Extremely fast and precise
Learning data needed	Can learn from a few examples	Usually needs many examples
Common sense	Rich, mostly automatic	Weak; must be learned or encoded
Generality	Very general across tasks	Mostly narrow, task-specific
Fatigue and emotion	Gets tired; influenced by emotion	No fatigue; no genuine emotion
Memory	Limited, can forget	Large, exact storage
Explanation	Can usually explain reasons	Many models are hard to explain

What is Artificial Intelligence?

The term **artificial intelligence** was coined by John McCarthy in the 1955 proposal for the Dartmouth summer research project, held in 1956. McCarthy later described AI as "the science and engineering of making intelligent machines, especially intelligent computer programs."

Many textbook authors have offered definitions. Russell and Norvig, in their standard textbook *Artificial Intelligence: A Modern Approach*, collected eight well-known definitions and showed that they fall along two dimensions: whether they are about **thought processes** or **behaviour**, and whether they measure success against **human performance** or against an ideal standard of **rationality**.

Table 1.4: Classic definitions of AI grouped by approach

Author (year)	Definition (short form)	Approach
Haugeland (1985)	"The exciting new effort to make computers think … machines with minds"	Thinking humanly
Bellman (1978)	Automation of activities we associate with human thinking: decision-making, problem solving, learning	Thinking humanly
Charniak and McDermott (1985)	"The study of mental faculties through the use of computational models"	Thinking rationally

Author (year)	Definition (short form)	Approach
Winston (1992)	"The study of the computations that make it possible to perceive, reason, and act"	Thinking rationally
Kurzweil (1990)	Creating machines that perform functions requiring intelligence when performed by people	Acting humanly
Rich and Knight (1991)	"The study of how to make computers do things at which, at the moment, people are better"	Acting humanly
Poole et al. (1998)	"Computational Intelligence is the study of the design of intelligent agents"	Acting rationally
Nilsson (1998)	AI "is concerned with intelligent behavior in artifacts"	Acting rationally

Exam Tip: For a 2-mark "Define AI" question, quote one definition (Rich and Knight is the easiest to remember) and add one line: "AI is the branch of computer science concerned with building machines that perceive, reason, learn and act intelligently." Mentioning McCarthy and 1956 earns credit.

The Four Approaches to AI

Combining the two dimensions gives four approaches. Each has been pursued seriously, by different people with different methods.

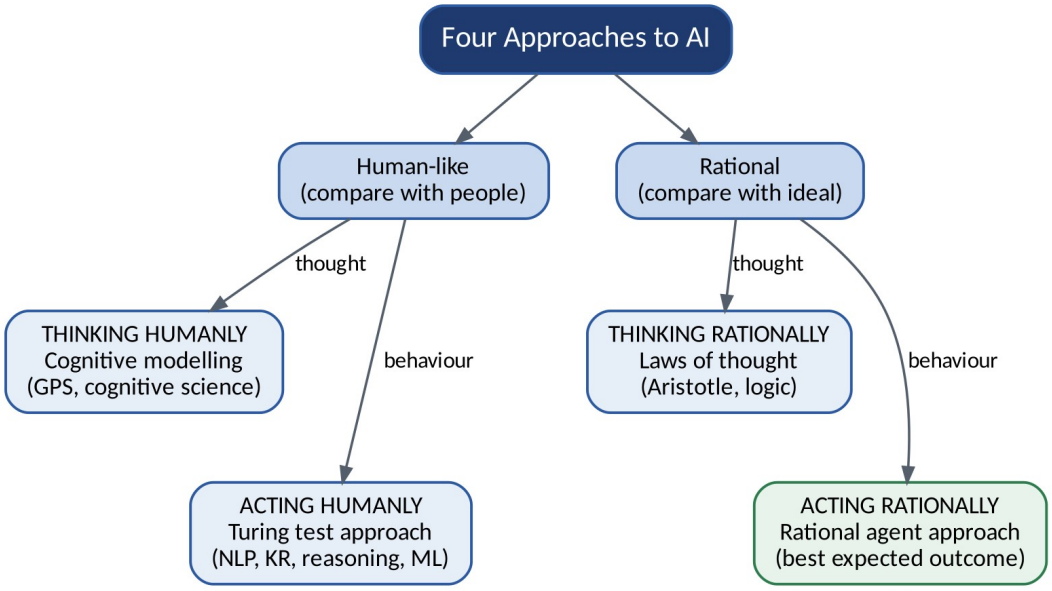


Figure 1.2: The four approaches to AI

Acting Humanly: The Turing Test Approach

In his 1950 paper "Computing Machinery and Intelligence", Alan Turing replaced the vague question "Can machines think?" with a practical test, which he called the **imitation game** and which we now call the **Turing test**.

In the test, a human **interrogator** communicates in writing (by teletype, in Turing's day) with two hidden players: one a human, the other a computer. The interrogator asks any questions they like. If, after the conversation, the interrogator cannot reliably tell which player is the computer, the computer is said to have passed the test.

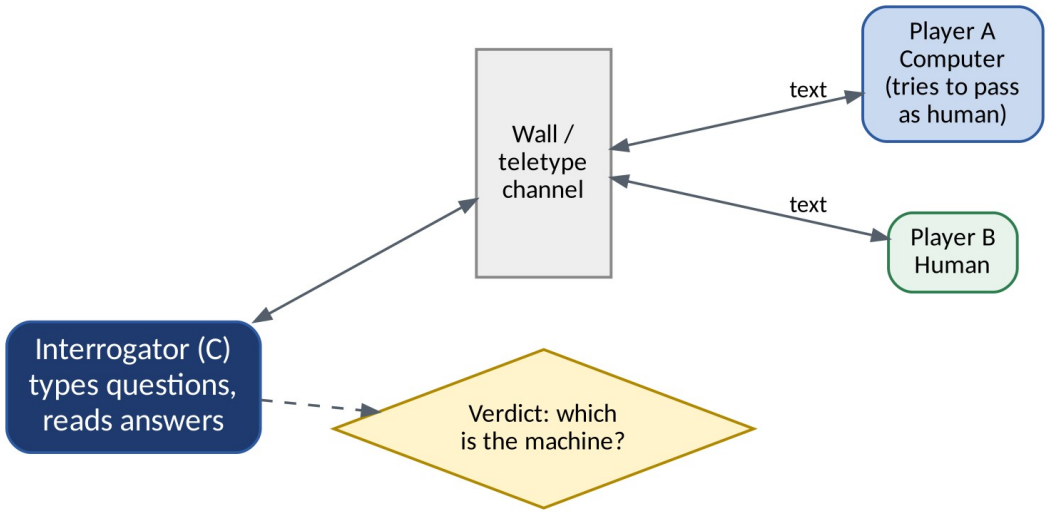


Figure 1.3: The Turing test (imitation game)

To pass the Turing test a computer would need four capabilities, which together cover most of AI:

1. **Natural language processing** — to communicate successfully in English (or another language).
2. **Knowledge representation** — to store what it knows or hears.
3. **Automated reasoning** — to use the stored information to answer questions and draw new conclusions.
4. **Machine learning** — to adapt to new circumstances and detect patterns.

Turing deliberately avoided physical interaction, because physically simulating a person is not needed for intelligence. The **total Turing test** adds a video signal, so the interrogator can test perceptual abilities, and a hatch through which objects can be passed. To pass it, the computer additionally needs:

5. **Computer vision** — to perceive objects.
6. **Robotics** — to manipulate objects and move about.

These six disciplines make up most of AI.

Strengths and criticisms of the Turing test. Its strength is that it is objective and behavioural: we do not need to agree on a definition of thinking. Its criticisms are that it tests only human-like behaviour (a program might be very intelligent but obviously non-human); it can be "gamed" by tricks, evasive answers and deliberate typing errors; and it says nothing about understanding. Researchers in AI have in fact spent little effort trying to pass the test, preferring to study the underlying principles — just as aeronautical engineers stopped trying to build machines that fly "so exactly like pigeons that they can fool even other pigeons."

Worked Example — ELIZA and the Turing test: In the mid-1960s Joseph Weizenbaum at MIT wrote ELIZA, a program that imitated a psychotherapist by matching keywords and reflecting the user's sentences back as questions ("I feel sad" → "Why do you feel sad?"). Some users became emotionally attached to it, even though ELIZA had no understanding at all. The lesson for the exam: appearing human in a short

conversation is not the same as being intelligent, which is exactly the weakness critics point out in the Turing test.

Thinking Humanly: The Cognitive Modelling Approach

If we want to say a program thinks like a human, we must know how humans think. There are three ways to find out: introspection (observing our own thoughts), psychological experiments (observing people in action), and brain imaging (observing the brain at work). Once there is a precise theory of the mind, it can be expressed as a computer program; if the program's input–output behaviour and timing match human behaviour, that is evidence that some of the program's mechanisms may also be operating in humans.

Newell and Simon's **General Problem Solver (GPS)** (1957 onwards) is the classic example. They were less interested in whether GPS solved problems correctly and more interested in whether its reasoning steps matched the steps reported by human subjects solving the same problems. This approach led to the interdisciplinary field of **cognitive science**, which combines computer models from AI with experimental techniques from psychology.

Thinking Rationally: The Laws of Thought Approach

The Greek philosopher Aristotle tried to codify "right thinking" — irrefutable reasoning processes. His **syllogisms** give patterns of argument that always yield correct conclusions from correct premises, for example "Socrates is a man; all men are mortal; therefore Socrates is mortal." These "laws of thought" started the field of **logic**.

By 1965, programs existed that could, in principle, solve any solvable problem stated in logical notation. The **logicist** tradition in AI hopes to build intelligent systems on such programs. Two obstacles remain: it is hard to take informal, uncertain knowledge and state it in formal logic, and solving a problem "in principle" is very different from solving it in practice — even small problems can exhaust computing resources without guidance about which reasoning steps to try first.

Acting Rationally: The Rational Agent Approach

An **agent** is anything that acts (from the Latin *agere*, to do). A computer agent is expected to do more than just run: it should operate autonomously, perceive its environment, persist over a long time, adapt to change, and create and pursue goals. A **rational agent** is one that acts so as to achieve the best outcome or, when there is uncertainty, the best **expected** outcome.

The rational-agent approach is more general than the laws-of-thought approach, because correct inference is only one way of acting rationally; reflexes (such as pulling your hand off a hot stove) can be rational without any deliberation. It is also more amenable to scientific development than approaches based on human behaviour, because the standard of rationality is mathematically well defined. For these reasons the rational-agent view is the one taken by most modern textbooks, and it is the one we use for problem solving later in this unit.

Table 1.5: The four approaches to AI compared

Approach	Question asked	Method / example	Main limitation
Thinking humanly	Does it think the way people do?	Cognitive modelling; GPS; cognitive science	Need a working theory of the human mind
Thinking rationally	Does it think correctly?	Logic, syllogisms, theorem provers	Hard to formalise uncertain knowledge; slow in practice
Acting humanly	Does it behave like a person?	Turing test; NLP, KR, reasoning, ML	Human-likeness is not the same as intelligence

Approach	Question asked	Method / example	Main limitation
Acting rationally	Does it do the right thing?	Rational agents maximising expected outcome	Perfect rationality is often computationally infeasible

Memory Trick: "HT-RT-HA-RA" — Human Thinking (cognitive science), Rational Thinking (logic), Human Acting (Turing test), Rational Acting (agents). The top row is about the mind, the bottom row about behaviour; the left column compares with people, the right column with an ideal.

The Chinese Room Argument

In 1980 the philosopher John Searle published a thought experiment designed to show that passing a behavioural test like the Turing test does not prove that a machine **understands** anything.

Imagine a person who knows no Chinese, locked in a room. Through a slot they receive cards with Chinese characters (questions). They have a large rulebook, written in English, that tells them which Chinese symbols to write in reply, purely by matching shapes. They pass the replies back out. To the Chinese speakers outside, the answers are perfect; the room seems to understand Chinese. Yet the person inside understands nothing — they are only manipulating symbols according to rules.

Searle's point is that a computer running a program is in the same position as the person in the room: it manipulates symbols (**syntax**) without grasping their meaning (**semantics**). Therefore, he argued, running the right program is not by itself sufficient for a mind.

The main replies to Searle are:

- **Systems reply** — the person does not understand Chinese, but the whole system (person + rulebook + room) does.
- **Robot reply** — if the program were put in a robot that perceived and acted in the world, its symbols would be grounded in experience and it would understand.
- **Brain simulator reply** — if the program simulated the actual neurons of a Chinese speaker's brain, it would understand just as the speaker does.

Searle rejected each reply. The debate is still open, and it is closely linked to the distinction between strong and weak AI.

Key Point: The Turing test asks whether a machine can behave intelligently; the Chinese room asks whether behaving intelligently is enough for genuine understanding. Turing focuses on behaviour, Searle on the inner meaning of the symbols.

Strong AI and Weak AI

The terms were introduced by Searle in the same 1980 discussion.

- **Weak AI** is the claim that machines can act **as if** they were intelligent. A weak-AI system is a useful tool for studying the mind or for doing tasks; nobody claims it really has a mind.
- **Strong AI** is the claim that a suitably programmed machine **actually has** a mind — it really thinks and understands, rather than merely simulating thinking.

Separately, AI systems are often classified by the **breadth** of what they can do:

- **Narrow AI** (Artificial Narrow Intelligence, ANI) performs one task or a small set of tasks: playing chess, recognising faces, recommending films, translating text. All deployed AI today is narrow, even when very powerful.

- **General AI** (Artificial General Intelligence, AGI) would match human ability across practically any intellectual task, transferring knowledge between domains. It does not yet exist and its timing is debated.
- **Super AI** (Artificial Superintelligence, ASI) is a hypothetical system that would exceed the best human performance in almost every domain.

Table 1.6: Strong vs weak AI and narrow vs general AI

Basis	Category	Meaning	Example / status
Philosophical claim	Weak AI	Machine simulates intelligent behaviour	Chess programs, chatbots as tools
Philosophical claim	Strong AI	Machine genuinely has a mind and understanding	Hypothetical; disputed (Chinese room)
Capability	Narrow AI (ANI)	Excellent at one specific task	Spam filter, face unlock, AlphaGo
Capability	General AI (AGI)	Human-level across most tasks	Not yet achieved
Capability	Super AI (ASI)	Beyond best humans in nearly all areas	Purely hypothetical

Exam Tip: Students often mix up the two pairs. "Strong versus weak" is about whether the machine really thinks (a philosophical question). "Narrow versus general" is about how many different tasks it can handle (an engineering question). Many textbooks use "weak AI" loosely to mean narrow AI; if you mention this, state both meanings clearly.

Summary

- Intelligence is the ability to learn, reason, solve problems, perceive, use language and adapt; Gardner identified multiple forms of intelligence.
- AI was named by John McCarthy for the 1956 Dartmouth workshop; definitions vary along two dimensions — thought versus behaviour, human versus rational.
- The four approaches are thinking humanly (cognitive modelling), thinking rationally (laws of thought), acting humanly (Turing test) and acting rationally (rational agents). The rational-agent approach is the modern standard.
- The Turing test (1950) needs NLP, knowledge representation, automated reasoning and machine learning; the total Turing test adds vision and robotics.
- Searle's Chinese room (1980) argues that symbol manipulation (syntax) is not understanding (semantics).
- Weak AI simulates intelligence; strong AI would really have a mind. Narrow AI handles one task; general AI would handle most human tasks.

Foundations of Artificial Intelligence

AI did not appear from nowhere in 1956. It drew on ideas that had been developing for more than two thousand years in several older disciplines. Each discipline asked questions that AI later needed answered: How can valid conclusions be drawn? What can be computed? How should we decide under uncertainty? How does the brain process information? How can machines regulate themselves? How does language relate to thought?

Knowing these foundations helps you understand why AI uses the tools it does — logic, probability, utility, neural networks, feedback control and grammars — and it is a common 5-mark and 10-mark exam question.

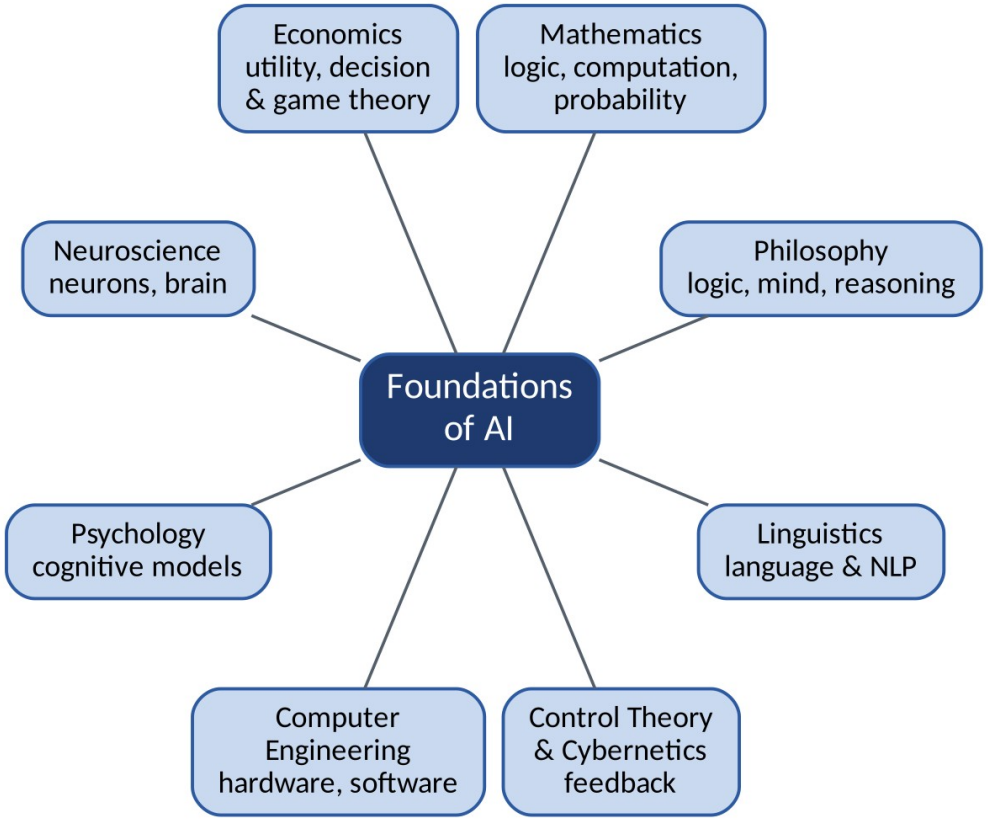


Figure 1.4: The eight disciplines that form the foundations of AI

Philosophy

Philosophy asked: Can formal rules be used to draw valid conclusions? How does the mind arise from a physical brain? Where does knowledge come from? How does knowledge lead to action?

- **Aristotle** (384–322 BC) formulated a precise set of laws governing the rational part of the mind. His syllogisms allowed conclusions to be generated mechanically from premises — the first step towards logic-based AI.
- **Ramon Llull** (around 1300) and later **Thomas Hobbes** suggested that reasoning was like numerical computation — "we add and subtract in our silent thoughts."

- **René Descartes** (1596–1650) gave the first clear discussion of the distinction between mind and matter (**dualism**), and of the problems it raises. The opposite view, **materialism**, holds that the brain's operation according to the laws of physics *constitutes* the mind.
- The **empiricist** movement, starting with Francis Bacon and John Locke ("Nothing is in the understanding, which was not first in the senses"), held that knowledge comes from experience. **David Hume** proposed the principle of **induction**: general rules are acquired by exposure to repeated associations — the philosophical root of machine learning.
- **Rudolf Carnap** and the Vienna Circle developed **logical positivism**, in which all knowledge can be characterised by logical theories connected to observation sentences.
- Philosophy also connected knowledge to action: Aristotle argued that we deliberate about means, not ends — an idea that reappears in goal-based agents and planning.

Mathematics

Mathematics asked: What are the formal rules to draw valid conclusions? What can be computed? How do we reason with uncertain information?

- **Logic.** George Boole (1815–1864) worked out propositional (Boolean) logic. Gottlob Frege (1848–1925) extended it to first-order logic with objects and relations, still used for knowledge representation today.
- **Algorithms and computability.** The first non-trivial algorithm is often said to be Euclid's algorithm for the greatest common divisor. Kurt Gödel's **incompleteness theorem** (1931) showed that in any sufficiently powerful formal theory there are true statements that cannot be proved. Alan Turing (1936) characterised exactly which functions are **computable** using the Turing machine, and showed that some functions — such as deciding whether an arbitrary program halts — are not computable at all.
- **Tractability.** Even computable problems may be **intractable**: if the time to solve them grows exponentially with the size of the instance. The theory of **NP-completeness** (Steven Cook, 1971; Richard Karp, 1972) gives a way to recognise such problems. This is why AI must use heuristics and approximations — and why search strategy matters so much.
- **Probability.** Gerolamo Cardano first framed the idea of probability in terms of gambling outcomes; Pascal and Fermat developed it further; Thomas Bayes (1702–1761) proposed the rule for updating probabilities in the light of new evidence. **Bayes' rule** underlies most modern approaches to uncertain reasoning in AI.

Economics

Economics asked: How should we make decisions so as to maximise payoff? How should we do this when others may not go along? How should we do this when the payoff may be far in the future?

- Adam Smith (1776) treated economies as consisting of individual agents maximising their own economic well-being.
- The mathematical notion of **utility** — a numerical measure of preferred outcomes — was formalised in **decision theory**, which combines probability theory with utility theory to make rational decisions under uncertainty.
- John von Neumann and Oskar Morgenstern's *Theory of Games and Economic Behavior* (1944) founded **game theory**, which covers decisions in the presence of other agents — the basis for multi-agent AI and game playing.

- **Operations research**, which grew out of wartime radar and logistics work, studied sequential decisions; Richard Bellman's **Markov decision processes** formalised a class of sequential decision problems used in reinforcement learning.
- Herbert Simon showed that real decision makers **satisfice** — make decisions that are "good enough" — rather than laboriously computing optimal ones. He won the Nobel Prize in Economics in 1978 for this work, and he was also one of the founders of AI.

Neuroscience

Neuroscience asked: How do brains process information?

- Paul Broca's study of patients with brain damage (1861) showed that specific areas of the brain are responsible for specific cognitive functions — in particular, speech production is localised in an area of the left hemisphere now called Broca's area.
- Camillo Golgi's staining technique allowed individual **neurons** to be observed; Santiago Ramón y Cajal used it in his pioneering studies of the brain's neuronal structures.
- The brain contains roughly 10^11 neurons (a hundred billion) connected by far more synapses. A collection of simple cells can lead to thought, action and consciousness — "brains cause minds."
- Modern techniques such as EEG and functional MRI let researchers observe the brain at work. Neuroscience inspired **artificial neural networks**, starting with McCulloch and Pitts' model neuron in 1943.

Psychology

Psychology asked: How do humans and animals think and act?

- Hermann von Helmholtz and Wilhelm Wundt applied the scientific method to human vision and thought; Wundt opened the first laboratory of experimental psychology in Leipzig in 1879.
- **Behaviourism** (John Watson and others) rejected any theory involving mental processes and studied only objective measures of stimuli and responses. It worked well for animals but less well for humans.
- **Cognitive psychology** views the brain as an information-processing device. Kenneth Craik's *The Nature of Explanation* (1943) specified three key steps of a knowledge-based agent: the stimulus is translated into an internal representation; the representation is manipulated by cognitive processes to derive new internal representations; and these are translated back into action.
- The meeting of AI and psychology produced **cognitive science**, often dated to a 1956 MIT symposium at which work by George Miller, Noam Chomsky, and Newell and Simon was presented.

Computer Engineering

Computer engineering asked: How can we build an efficient computer? For AI to succeed we need both intelligence and an artifact, and the computer has been the artifact of choice.

- Early electronic and electromechanical computers appeared during the Second World War: the code-breaking machines at Bletchley Park (including Colossus, 1943), Konrad Zuse's programmable Z-3 in Germany (1941), the Atanasoff–Berry Computer in the USA, and ENIAC at the University of Pennsylvania.
- Charles Babbage (1792–1871) had earlier designed the Difference Engine and the Analytical Engine, the latter a general-purpose programmable design. Ada Lovelace, who worked with Babbage, wrote programs for it and is often called the first programmer.

- AI has also contributed to computer science: time-sharing, interactive interpreters, linked lists, automatic storage management, and key concepts of symbolic, functional and object-oriented programming were pioneered or popularised in AI research. The programming language **Lisp** was created by John McCarthy in 1958 for AI work.
- Hardware improvements — faster processors, more memory and, more recently, massively parallel graphics processing units (GPUs) — have repeatedly made previously impractical AI methods feasible, most notably deep learning.

Control Theory and Cybernetics

Control theory asked: How can artifacts operate under their own control?

- Ktesibios of Alexandria (around 250 BC) built a water clock with a regulator that maintained a constant flow rate — the first known self-controlling machine.
- James Watt's centrifugal governor (18th century) regulated the speed of steam engines; Cornelis Drebbel invented the thermostat.
- Norbert Wiener's *Cybernetics* (1948) brought together control theory, communication and the idea that purposive behaviour arises from a **feedback** mechanism that tries to minimise the "error" between the current state and the goal state.
- Modern control theory designs systems that maximise an **objective function** over time — closely related to the AI idea of an agent maximising its performance measure. The main difference is that control theory leaned on calculus and continuous variables, whereas early AI used logic and symbolic reasoning to tackle problems such as language and planning that did not fit the continuous framework.

Linguistics

Linguistics asked: How does language relate to thought?

- In 1957 B. F. Skinner published *Verbal Behavior*, a behaviourist account of language learning. Noam Chomsky's review of the book pointed out that behaviourism could not explain creativity in language — how children understand and produce sentences they have never heard. Chomsky's own *Syntactic Structures* (1957) presented a formal theory of grammar precise enough, in principle, to be programmed.
- Modern **computational linguistics** or **natural language processing** combines linguistics with AI. Understanding language turned out to require understanding the subject matter and context, not just sentence structure — which links NLP closely with knowledge representation.

Table 1.7: Foundations of AI — disciplines, questions and contributions

Discipline	Key question for AI	Main contributions	Notable names
Philosophy	Can formal rules give valid conclusions? Where does knowledge come from?	Logic, mind–body problem, induction, reasoning and action	Aristotle, Descartes, Locke, Hume, Carnap
Mathematics	What can be computed? How to reason under uncertainty?	Formal logic, algorithms, computability, NP-completeness, probability	Boole, Frege, Gödel, Turing, Bayes
Economics	How to decide to maximise payoff?	Utility, decision theory, game theory, MDPs, satisficing	Smith, von Neumann, Morgenstern, Bellman, Simon

Discipline	Key question for AI	Main contributions	Notable names
Neuroscience	How do brains process information?	Neurons, brain localisation, inspiration for neural networks	Broca, Golgi, Cajal
Psychology	How do humans and animals think and act?	Behaviourism, cognitive psychology, cognitive science	Helmholtz, Wundt, Watson, Craik
Computer engineering	How to build efficient computers?	Programmable hardware, Lisp, GPUs	Babbage, Lovelace, Zuse, McCarthy
Control theory	How can artifacts control themselves?	Feedback, cybernetics, objective functions	Ktesibios, Watt, Wiener
Linguistics	How does language relate to thought?	Formal grammars, NLP	Skinner, Chomsky

Key Point: Each foundation supplied AI with a tool: philosophy and mathematics gave logic and reasoning; mathematics and economics gave probability, utility and decision making; neuroscience gave neural networks; psychology gave cognitive models; computer engineering gave the machine; control theory gave feedback and optimisation; linguistics gave grammars for language processing.

Memory Trick: "People Make Excellent New Plans; Computers Can Learn" — Philosophy, Mathematics, Economics, Neuroscience, Psychology, Computer engineering, Control theory, Linguistics.

Exam Tip: For a 10-mark question on foundations, draw Figure 1.4 first, then write one short paragraph per discipline with its key question, one or two contributions and one name. Eight neat paragraphs with correct names will score better than long paragraphs on only three disciplines.

Worked Example — Tracing foundations in a modern system: Consider a self-driving car. Its object detector is a deep neural network (neuroscience, computer engineering). It estimates the position of other cars with probabilistic filters (mathematics — Bayes' rule). It decides when to change lanes by weighing safety, speed and comfort (economics — utility and decision theory). It keeps the car in lane with a feedback steering controller (control theory). It may accept spoken destinations (linguistics). And the very idea that a machine could "decide" draws on philosophy's account of reasoning and action. One product, all eight foundations.

Summary

- AI rests on eight foundations: philosophy, mathematics, economics, neuroscience, psychology, computer engineering, control theory and linguistics.
- Philosophy supplied logic, induction and the link between knowledge and action; mathematics formalised logic, computation, tractability and probability.
- Economics contributed utility, decision theory and game theory; Simon's satisficing links economics and AI.
- Neuroscience revealed neurons and inspired neural networks; psychology contributed cognitive models and cognitive science.
- Computer engineering provided the hardware and languages such as Lisp; control theory contributed feedback and objective functions; linguistics contributed formal grammars and NLP.

History and Applications of AI

The history of AI is a story of bold predictions, real breakthroughs, disappointments and comebacks. Twice, enthusiasm ran ahead of results and funding collapsed — the periods now called **AI winters**. Each comeback was driven by a new idea (knowledge-based systems, then statistical learning, then deep learning) together with more powerful hardware and more data.

For the exam you need the main phases in the right order, a handful of exact dates, and the reason each phase rose or fell.

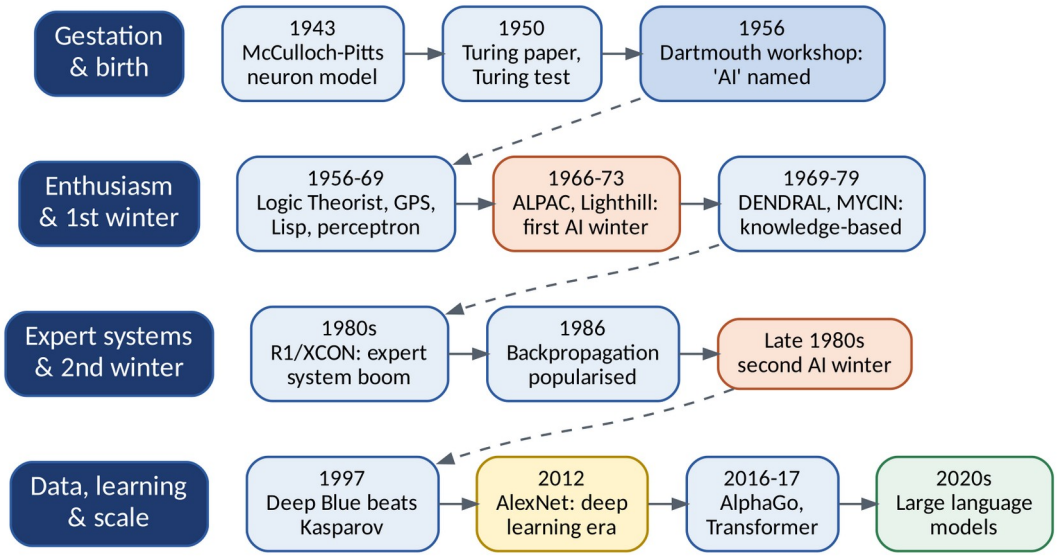


Figure 1.5: Timeline of the history of AI

The Gestation of AI (1943–1955)

The first work now recognised as AI was done by **Warren McCulloch and Walter Pitts** in **1943**. Drawing on the physiology of neurons, propositional logic and Turing's theory of computation, they proposed a model of artificial neurons in which each neuron is either "on" or "off". They showed that any computable function could be computed by some network of connected neurons, and that logical connectives (AND, OR, NOT) could be implemented by simple networks.

In **1949 Donald Hebb** proposed a simple updating rule for modifying the connection strengths between neurons — now called **Hebbian learning** — which remains influential. In **1950** two Harvard students, **Marvin Minsky and Dean Edmonds**, built SNARC, the first neural network computer, which used vacuum tubes to simulate a network of 40 neurons.

Also in **1950**, **Alan Turing** published "Computing Machinery and Intelligence", which introduced the Turing test, machine learning, genetic algorithms and reinforcement learning as ideas. Turing's vision set the agenda for the field before it even had a name.

The Birth of AI (1956)

John McCarthy, then at Dartmouth College, persuaded **Marvin Minsky, Claude Shannon and Nathaniel Rochester** to help him bring together US researchers interested in automata theory, neural nets and the study of intelligence. They organised a two-month workshop at Dartmouth in the summer of **1956**. The proposal stated the conjecture that "every aspect of learning or any other feature of intelligence can in

principle be so precisely described that a machine can be made to simulate it." It is in this proposal that the name **artificial intelligence** first appears.

The star of the workshop was the **Logic Theorist** of Allen Newell and Herbert Simon, a reasoning program that could prove theorems in Whitehead and Russell's *Principia Mathematica*. The Dartmouth workshop did not produce new breakthroughs itself, but it introduced the major figures to each other, and for the next twenty years the field was dominated by these people and their students at MIT, Carnegie Mellon, Stanford and IBM.

> **Key Point:** 1956, Dartmouth College, John McCarthy — the birth and naming of AI. If you remember only one date from this chapter, remember this one.

Early Enthusiasm and Great Expectations (1952–1969)

The early years were full of successes, considering how primitive computers were. Researchers kept showing that computers could do things people had said "a machine can never do".

- **General Problem Solver (GPS)** (Newell and Simon, from 1957) imitated human problem-solving protocols; it was probably the first program to embody the "thinking humanly" approach.
- **Arthur Samuel** at IBM wrote checkers (draughts) programs from 1952 onwards that learned to play at a strong amateur level — and better than Samuel himself — disproving the idea that computers can do only what they are told.
- **Herbert Gelernter** built the Geometry Theorem Prover (1959).
- **John McCarthy** (now at MIT) in **1958** defined the high-level language **Lisp**, which became the dominant AI programming language for the next 30 years, and described the Advice Taker, a hypothetical program embodying general world knowledge.
- **Frank Rosenblatt** introduced the **perceptron** (1958), a learning neural network, and later proved the perceptron convergence theorem.
- **Minsky's students** studied limited problems in "microworlds", most famously the **blocks world** — a set of solid blocks on a tabletop that a robot hand rearranges. Terry Winograd's SHRDLU (around 1971) understood English commands about this world.
- **J. A. Robinson** discovered the **resolution** method (1965), a complete inference procedure for first-order logic.
- **Joseph Weizenbaum's ELIZA** (mid-1960s) imitated a psychotherapist using simple pattern matching.
- **Shakey** (SRI, 1966–1972) was the first mobile robot to reason about its own actions, combining perception, planning and action.

Herbert Simon predicted in 1957 that within ten years a computer would be chess champion. In fact it took about forty.

A Dose of Reality: The First AI Winter (1966–1973)

The early programs worked on simple examples but failed on wider or harder problems, for three main reasons:

1. **No real knowledge.** Early programs knew nothing of their subject matter and relied on simple syntactic manipulation. Machine translation was the famous failure: the 1966 **ALPAC report** to the US government concluded that there was no machine translation of general scientific text and none

in immediate prospect, and US government funding for academic translation projects was cancelled.

2. **Intractability.** Programs solved problems by trying combinations of steps. This works in microworlds with few objects but explodes combinatorially on real problems. Before NP-completeness theory, many believed that "scaling up" was just a matter of faster hardware.
3. **Limits of the representations.** Minsky and Papert's book *Perceptrons* (1969) proved that a single-layer perceptron could not represent some simple functions (for example exclusive-OR). Although the results did not apply to multi-layer networks, research funding for neural-net research dwindled almost to nothing.

In the UK, the **Lighthill report** (1973) criticised the failure of AI to achieve its "grandiose objectives", and the British government ended support for AI research in all but a few universities. This period of reduced funding and interest is called the **first AI winter**.

Knowledge-Based Systems (1969–1979)

The response was to use domain-specific knowledge in larger reasoning steps — "to solve a hard problem, you almost have to know the answer already."

- **DENDRAL** (Stanford; Buchanan, Feigenbaum and Lederberg) inferred molecular structure from mass-spectrometer data. It was the first successful **knowledge-intensive** system: its expertise came from large numbers of special-purpose rules.
- **MYCIN** (Edward Shortliffe and colleagues) diagnosed blood infections using about 450 rules and performed as well as some experts and considerably better than junior doctors. It introduced **certainty factors** to handle uncertainty.
- These successes established the **expert systems** paradigm and the importance of **knowledge representation**; languages such as **Prolog** (1972) and frame-based representations were developed.

AI Becomes an Industry; the Second Winter (1980–late 1980s)

The first successful commercial expert system, **R1** (later called **XCON**), began operation at Digital Equipment Corporation in 1982. It configured orders for new computer systems and was reported to save the company large sums every year. By the late 1980s almost every major US corporation had its own AI group using or investigating expert systems.

In 1981 Japan announced the **Fifth Generation** project, a ten-year plan to build intelligent computers running Prolog. The USA and UK responded with their own large research programmes. The AI industry boomed from a few million dollars in 1980 to billions of dollars by 1988.

Soon after came a period of collapse. Many companies failed to deliver on extravagant promises; expert systems proved expensive to build and maintain, brittle outside their narrow domain and unable to learn. The market for specialised **Lisp machines** collapsed in 1987 as cheaper general-purpose workstations caught up. This bust is called the **second AI winter**.

The Return of Neural Networks (1986–present)

In the mid-1980s several groups reinvented the **back-propagation** learning algorithm, first found in 1969 by Bryson and Ho. The 1986 collection *Parallel Distributed Processing* (Rumelhart, McClelland and others), and in particular the paper by **Rumelhart, Hinton and Williams**, made back-propagation widely known. This revived **connectionist** (neural network) research as an alternative to symbolic AI.

AI Adopts the Scientific Method; Probabilistic Reasoning (1987–present)

From the late 1980s AI became more rigorous: claims were based on theorems or careful experiments on shared benchmark data rather than on intuition. **Hidden Markov models** came to dominate speech recognition. **Judea Pearl's** *Probabilistic Reasoning in Intelligent Systems* (1988) introduced **Bayesian networks**, giving a principled way to represent and reason with uncertain knowledge. Machine learning and data mining became distinct, statistically grounded fields.

Intelligent Agents and Big Data (1995–2011)

Researchers returned to the "whole agent" problem — systems that perceive, reason and act — encouraged by the growth of the Internet (search engines, recommender systems and web "bots"). Very large data sets became available, and it was observed that, for many tasks, more data matters more than a cleverer algorithm.

- **1997**: IBM's **Deep Blue** defeated world chess champion **Garry Kasparov** in a six-game match.
- **2005**: Stanford's robot car Stanley won the DARPA Grand Challenge, driving autonomously across about 212 km of desert.
- **2011**: IBM's **Watson** defeated human champions on the quiz show *Jeopardy!*.

Deep Learning (2012–present)

Deep learning uses neural networks with many layers. Although the ideas were old, three things came together around 2010: huge labelled data sets (such as **ImageNet**), fast parallel hardware (**GPUs**), and improved training techniques.

- **2012**: **AlexNet** (Krizhevsky, Sutskever and Hinton), a deep convolutional network trained on GPUs, won the ImageNet image-classification competition by a large margin. This is usually taken as the start of the deep-learning era.
- **2014**: Generative adversarial networks (Goodfellow and colleagues) made realistic image generation possible.
- **2016**: DeepMind's **AlphaGo** defeated top Go player **Lee Sedol** 4–1, combining deep networks with tree search and reinforcement learning.
- **2017**: The **Transformer** architecture ("Attention Is All You Need", Vaswani and colleagues) replaced recurrent networks for language tasks and became the basis of modern language models.
- **2018**: Yoshua Bengio, Geoffrey Hinton and Yann LeCun received the ACM Turing Award (for 2018) for their work on deep learning.
- **2020**: DeepMind's **AlphaFold 2** produced highly accurate protein-structure predictions.

Large Language Models and Generative AI (2018–present)

A **large language model (LLM)** is a Transformer-based neural network trained on enormous amounts of text to predict the next word (token). Scaling up the model and the data turned out to produce surprisingly general abilities: answering questions, summarising, translating, writing code and reasoning step by step.

- **2018**: BERT (Google) and the first GPT model (OpenAI) showed the power of pre-training on large text corpora followed by fine-tuning.
- **2020**: GPT-3, with 175 billion parameters, showed that very large models could perform new tasks from a few examples in the prompt.
- **November 2022**: OpenAI released **ChatGPT**, bringing conversational LLMs to the general public.

- **2023 onwards**: Many competing general-purpose assistants and multimodal models (handling text, images and audio) appeared from several companies and open-source communities, and AI regulation became a major policy topic.
- **2024**: The Nobel Prize in Physics went to John Hopfield and Geoffrey Hinton for foundational work on neural networks, and part of the Nobel Prize in Chemistry went to Demis Hassabis and John Jumper for AlphaFold.

Despite these advances, today's systems are still **narrow** in the philosophical sense discussed in the previous chapter: they can be fluent yet make factual errors ("hallucinations"), and debates about understanding echo Searle's Chinese room.

Table 1.8: Key milestones in the history of AI

Year / period	Milestone	Significance
1943	McCulloch–Pitts artificial neuron	First AI work; neural networks as logic
1949	Hebbian learning	First learning rule for neural connections
1950	Turing, "Computing Machinery and Intelligence"	Turing test; agenda for AI
1956	Dartmouth workshop; Logic Theorist	Birth and naming of AI
1957–58	GPS; Lisp; perceptron	Early enthusiasm; key tools
1965–66	Resolution; ELIZA; ALPAC report	Logic advances; translation failure
1969–73	*Perceptrons*; Lighthill report	First AI winter
1969–79	DENDRAL, MYCIN	Knowledge-based expert systems
1982	R1/XCON in commercial use	AI becomes an industry
1986	Back-propagation popularised	Neural networks return
1987–early 1990s	Lisp-machine collapse, expert-system bust	Second AI winter
1988	Bayesian networks (Pearl)	Probabilistic reasoning
1997	Deep Blue beats Kasparov	Search plus evaluation beats a world champion
2011	Watson wins *Jeopardy!*	Question answering in natural language
2012	AlexNet wins ImageNet	Start of deep-learning era
2016	AlphaGo beats Lee Sedol	Deep RL plus search masters Go
2017	Transformer architecture	Foundation of LLMs
2022	ChatGPT released	Generative AI reaches the public

Why the AI Winters Happened

Table 1.9: Causes of the two AI winters

Cause	First winter (around 1966–1973)	Second winter (late 1980s)
Over-promising	Predictions of chess champions and translation within a decade	Expert systems sold as general business solutions
Technical limit	Combinatorial explosion; single-layer perceptron limits	Brittleness; no learning; costly knowledge acquisition

Cause	First winter (around 1966–1973)	Second winter (late 1980s)
Trigger reports / events	ALPAC (1966), *Perceptrons* (1969), Lighthill (1973)	Collapse of Lisp-machine market (1987); Fifth Generation goals unmet
Recovery came from	Knowledge-based systems	Statistical learning, neural nets, probabilistic methods

Memory Trick: "Neurons, Name, Nice dreams, Night, Knowledge, Night again, Numbers, Networks, Next words" — 1943 neurons; 1956 the name; 1956–69 enthusiasm; first winter; expert-system knowledge; second winter; statistical numbers and data; deep networks in 2012; next-word prediction in LLMs.

Exam Tip: In a 10-mark "History of AI" answer, divide your answer into phases with headings (gestation, birth, enthusiasm, first winter, expert systems, second winter, statistical and agent era, deep learning, LLMs), and give two or three dated milestones per phase. Draw the timeline diagram. Examiners reward correct ordering and dates more than long descriptions.

Applications of AI

AI is now used in almost every sector. The table summarises typical applications; you should be able to give two or three examples for any sector named in a question.

Table 1.10: Applications of AI across sectors

Sector	Typical AI applications	Main AI techniques
Healthcare	Medical image analysis, diagnosis support, drug discovery, protein structure	Deep learning, expert systems
Finance and banking	Fraud detection, credit scoring, algorithmic trading, chatbots	Machine learning, anomaly detection
E-commerce	Product recommendation, dynamic pricing, search ranking	Recommender systems, learning to rank
Transport	Route finding, autonomous vehicles, traffic signal control	Search, computer vision, planning
Manufacturing	Predictive maintenance, quality inspection, robot assembly	Vision, robotics, time-series learning
Agriculture	Crop-disease detection, yield prediction, precision spraying	Image classification, drones
Education	Intelligent tutoring, automated grading, adaptive learning	NLP, knowledge tracing
Language	Machine translation, speech recognition, virtual assistants	Transformers, LLMs
Games	Chess, Go, video-game characters	Search, reinforcement learning
Security	Face recognition, intrusion detection, spam filtering	Vision, classification
Logistics	Warehouse robots, vehicle routing, demand forecasting	Search, optimisation, ML
Software engineering	Code completion, test generation, bug detection	LLMs

Worked Example — Route finding in a navigation app: When you ask a maps app for directions, the road network is modelled as a graph: junctions are states, road segments are actions, and travel time is the step cost. The app formulates the problem (start at your location, goal is the destination) and runs a search algorithm — a refined form of the uniform-cost and A* search studied in this unit and the next — to find the least-cost path. Live traffic data changes the step costs, so the route can be recomputed during the journey. This is the most common everyday use of the search techniques in this unit.

Summary

- 1943 McCulloch–Pitts neurons and 1950 Turing's paper mark the gestation of AI; the 1956 Dartmouth workshop (McCarthy, Minsky, Shannon, Rochester) gave it its name.
- Early enthusiasm (GPS, Samuel's checkers, Lisp, perceptron) was followed by the first AI winter (ALPAC 1966, *Perceptrons* 1969, Lighthill 1973) caused by lack of knowledge and combinatorial explosion.
- Knowledge-based expert systems (DENDRAL, MYCIN, R1/XCON) led to an AI industry in the 1980s, followed by the second AI winter in the late 1980s.
- Back-propagation (1986), probabilistic methods (1988) and big data revived the field; Deep Blue (1997) and Watson (2011) were public milestones.
- Deep learning began with AlexNet (2012); AlphaGo (2016), the Transformer (2017) and LLMs such as GPT-3 (2020) and ChatGPT (2022) followed.
- AI is applied in healthcare, finance, transport, manufacturing, agriculture, education, language, games and more.

Intelligent Agents and Problem Formulation

The second half of the unit is about **problem solving**: how a program can decide what to do when it faces a task it has not been explicitly programmed for. To talk about this precisely we first need the vocabulary of **intelligent agents**. Then we look at a special kind of agent — the **problem-solving agent** — and at how a problem must be **formulated** before it can be solved by search.

Intelligent Agents

An **agent** is anything that can be viewed as **perceiving** its environment through **sensors** and **acting** upon that environment through **actuators**.

- A human agent has eyes and ears as sensors, and hands, legs and voice as actuators.
- A robot has cameras and range finders as sensors, and motors as actuators.
- A software agent receives keystrokes, files and network packets as sensory inputs and acts by displaying output, writing files and sending packets.

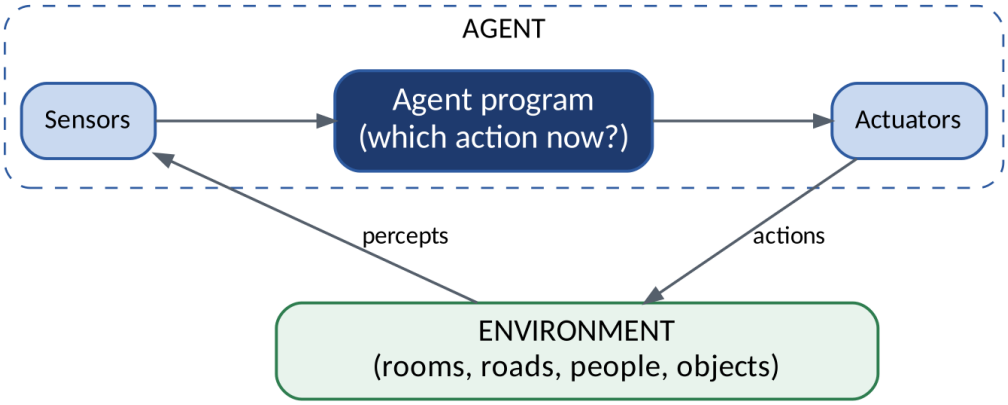


Figure 1.6: An agent interacts with its environment through sensors and actuators

Some key terms:

- **Percept** — the agent's perceptual input at a given instant.
- **Percept sequence** — the complete history of everything the agent has ever perceived.
- **Agent function** — a mathematical description that maps every possible percept sequence to an action.
- **Agent program** — the concrete implementation of the agent function, running on some physical **architecture**. In short: agent = architecture + program.

Rationality

A **rational agent** is one that does the right thing. Russell and Norvig define it as follows: for each possible percept sequence, a rational agent should select an action that is expected to maximise its **performance measure**, given the evidence provided by the percept sequence and whatever built-in knowledge the agent has.

What is rational at any time depends on four things: the performance measure that defines success; the agent's prior knowledge of the environment; the actions the agent can perform; and the agent's percept sequence to date. Rationality is not omniscience — a rational agent maximises *expected* performance and

cannot be blamed for failing to foresee something it could not perceive. A rational agent should, however, gather information (look before crossing the road), **learn** from what it perceives, and be **autonomous**, relying on its own percepts rather than only on its designer's prior knowledge.

Specifying the Task Environment: PEAS

The **task environment** is the "problem" to which a rational agent is the "solution". It is specified by four components, remembered by the acronym **PEAS**: **P**erformance measure, **E**nvironment, **A**ctuators, **S**ensors.

Table 1.11: PEAS descriptions of some agents

Agent	Performance measure	Environment	Actuators	Sensors
Automated taxi	Safe, fast, legal, comfortable trip; maximise profit	Roads, traffic, pedestrians, customers	Steering, accelerator, brake, signal, horn, display	Cameras, sonar, speedometer, GPS, odometer, engine sensors
Medical diagnosis system	Healthy patient, reduced costs	Patient, hospital, staff	Display of questions, tests, diagnoses, treatments	Keyboard entry of symptoms, findings, answers
Part-picking robot	Percentage of parts in correct bins	Conveyor belt with parts; bins	Jointed arm and hand	Camera, joint-angle sensors
Vacuum cleaner agent	Cleanliness, energy used, time	Rooms (squares A and B), dirt	Move left, move right, suck	Location sensor, dirt sensor
Route-finding app	Shortest or fastest route	Road network, traffic	Display route, voice directions	GPS, traffic feed, user input

Properties of Task Environments

Task environments are classified along several dimensions. These classifications determine which agent design, and which kind of search, is appropriate.

Table 1.12: Properties of task environments

Dimension	One extreme	Other extreme	Examples
Observability	Fully observable: sensors give the complete state	Partially observable (or unobservable)	Chess: fully; poker, taxi: partially
Number of agents	Single-agent	Multi-agent (competitive or cooperative)	Crossword: single; chess, taxi: multi
Determinism	Deterministic: next state fixed by current state and action	Stochastic (non-deterministic)	8-puzzle: deterministic; dice games, taxi: stochastic
Episodes	Episodic: each decision independent	Sequential: current decision affects future	Part sorting: episodic; chess: sequential
Change	Static: world does not change while agent thinks	Dynamic	Crossword: static; taxi: dynamic
State / time	Discrete: finite distinct states and actions	Continuous	Chess: discrete; taxi driving: continuous
Knowledge of rules	Known: outcomes of actions are given	Unknown: agent must learn how world works	Known board game vs new video game

The hardest case is partially observable, multi-agent, stochastic, sequential, dynamic, continuous and unknown — which describes driving a taxi. The problems solved by search in this unit are mostly at the easy end: fully observable, single-agent, deterministic, static, discrete and known.

Types of Agent Programs

Table 1.13: The five basic types of agent

Agent type	How it chooses actions	Example
Simple reflex agent	Condition–action rules on current percept only	Thermostat; "if dirty then suck"
Model-based reflex agent	Keeps internal state (a model) to handle partial observability	Car that tracks vehicles out of view
Goal-based agent	Considers future actions to reach a goal; uses search and planning	Route-finding agent
Utility-based agent	Chooses actions that maximise expected utility (degree of happiness)	Taxi trading off speed, safety, cost
Learning agent	Improves its components from experience using a critic and learning element	Spam filter that adapts to new spam

Key Point: A problem-solving agent is a goal-based agent. It looks ahead, considering sequences of actions that lead to desirable states, instead of reacting only to the current percept as a reflex agent does.

Problem-Solving Agents

A **problem-solving agent** decides what to do by finding a sequence of actions that leads to a desirable state. It uses an **atomic** representation: each state of the world is treated as a whole, with no internal structure visible to the problem-solving algorithm.

Consider the standard textbook example. An agent is on holiday in the city of Arad in Romania. Its flight leaves tomorrow from Bucharest. The agent therefore adopts the **goal** of being in Bucharest. Of the many ways it could spend the day, those that do not reach Bucharest in time can be rejected, which simplifies its decision problem.

The agent works in four phases:

1. **Goal formulation** — based on the current situation and its performance measure, the agent decides on a goal (a set of world states in which the goal is satisfied). Goals organise behaviour by limiting the objectives the agent is trying to achieve.
2. **Problem formulation** — the agent decides what actions and states to consider, given the goal. For the Romania trip, the states are "being in a particular town" and the actions are "driving from one town to a neighbouring town".
3. **Search** — the agent examines possible sequences of actions to find one that reaches the goal. A **search algorithm** takes a problem as input and returns a **solution** in the form of an action sequence.
4. **Execution** — the agent carries out the recommended actions one by one.

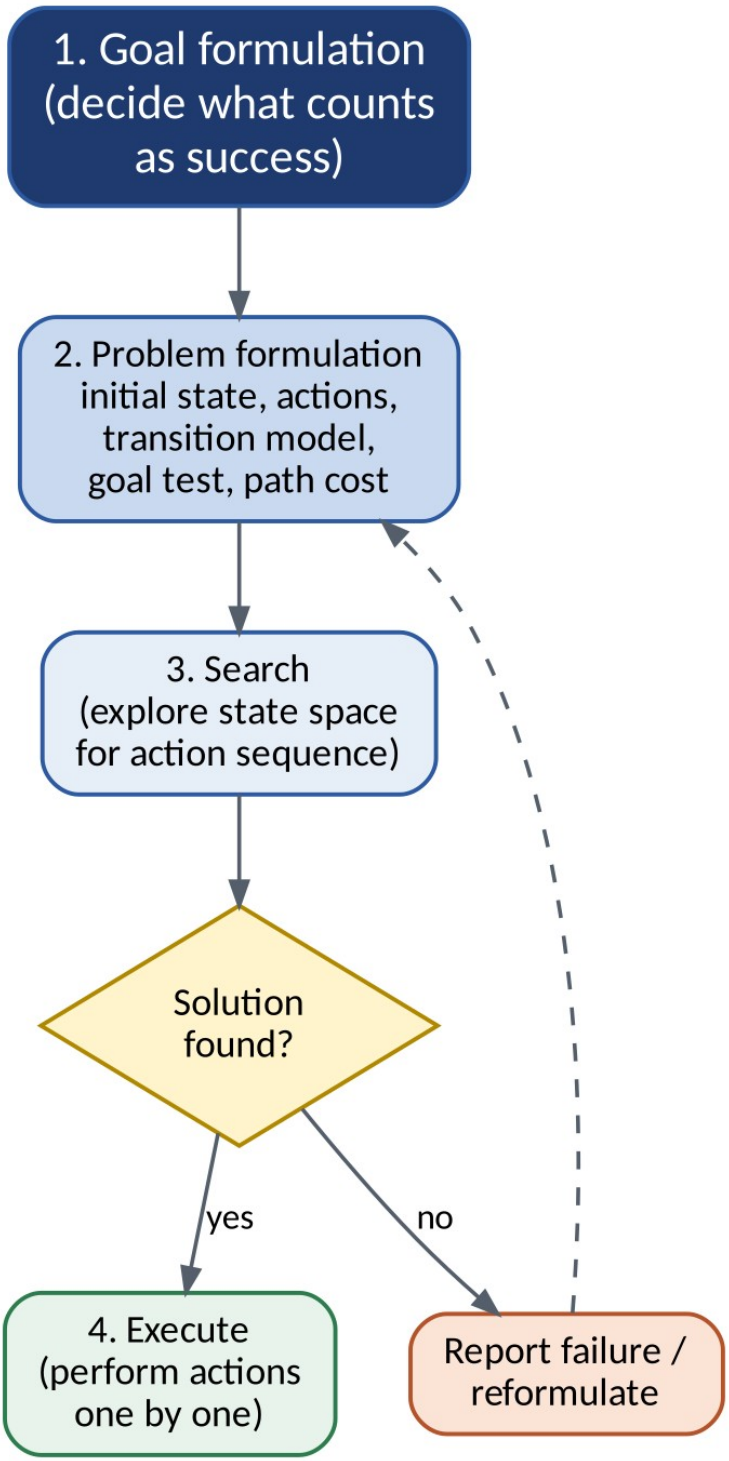


Figure 1.7: The four phases of a problem-solving agent

The design is "formulate, search, execute". While executing, a simple problem-solving agent ignores its percepts, because it knows in advance what will happen: this is called an **open-loop** system. Open-loop execution is safe only if the environment is fully observable, deterministic and known.

```
function SIMPLE-PROBLEM-SOLVING-AGENT(percept) returns an action
  persistent: seq, an action sequence, initially empty
              state, a description of the current world state
              goal, a goal, initially null
```

```
            problem, a problem formulation
  state <- UPDATE-STATE(state, percept)
  if seq is empty then
      goal    <- FORMULATE-GOAL(state)
      problem <- FORMULATE-PROBLEM(state, goal)
      seq     <- SEARCH(problem)
      if seq = failure then return a null action
  action <- FIRST(seq)
  seq    <- REST(seq)
  return action
```

Formulating Problems

A **well-defined problem** can be specified formally by five components. Formulating a problem means choosing each of them carefully.

1. **Initial state** — the state the agent starts in. Example: In(Arad).
2. **Actions** — a description of the possible actions available. Given a state s, ACTIONS(s) returns the set of actions that can be executed in s. Each such action is said to be **applicable** in s. Example: from In(Arad), the applicable actions are {Go(Sibiu), Go(Timisoara), Go(Zerind)}. In older textbooks and in production-system language, actions are called **operators**.
3. **Transition model** — a description of what each action does, given by a function RESULT(s, a) that returns the state reached by doing action a in state s. That state is called a **successor** of s. Example: RESULT(In(Arad), Go(Zerind)) = In(Zerind). Older texts combine actions and transition model into a single **successor function** S(x) that returns a set of (action, successor) pairs.
4. **Goal test** — determines whether a given state is a goal state. Sometimes the goal is an explicit set of states ({In(Bucharest)}); sometimes it is an abstract property (checkmate in chess, "no two queens attack each other").
5. **Path cost** — a function that assigns a numeric cost to each path. The agent chooses a cost function that reflects its own performance measure. Usually the path cost is the sum of the **step costs** of the individual actions along the path, written c(s, a, s′). Example: distance in kilometres.

Together, the initial state, actions and transition model implicitly define the **state space** of the problem. A **path** in the state space is a sequence of states connected by a sequence of actions. A **solution** is an action sequence that leads from the initial state to a goal state. Solution quality is measured by the path cost; an **optimal solution** has the lowest path cost among all solutions.

Table 1.14: The five components of a problem formulation

Component	Meaning	Romania route-finding example
Initial state	Where the agent starts	In(Arad)
Actions / operators	What can be done in state s	Go(neighbouring town)
Transition model	Result of action a in state s	RESULT(In(Arad), Go(Sibiu)) = In(Sibiu)
Goal test	Is this a goal state?	Is the state In(Bucharest)?
Path cost	Numeric cost of a path	Sum of road distances in km

Abstraction

Real-world situations contain enormous detail: the weather, the scenery, the radio station, the traveling companions. A formulation keeps only what matters for the goal; the process of removing detail is called **abstraction**. An abstraction is **valid** if any abstract solution can be expanded into a solution in the more

detailed world, and **useful** if carrying out each abstract action is easier than the original problem. "Drive from Arad to Sibiu" is a valid and useful abstract action: any driver can carry it out without further planning.

> **Exam Tip:** Whenever a question says "formulate the problem" (water jug, 8-puzzle, missionaries and cannibals, vacuum world), write the five headings — states, initial state, actions/operators, transition model, goal test, path cost — and fill in each for that problem. This simple structure alone earns most of the marks.

States and Operators

A **state** is a description of the world (or of the relevant part of it) at one moment. Choosing a good state representation is the most important design decision in formulating a problem. It should contain everything needed to decide which actions are possible and whether the goal is reached — and nothing more. For example:

- In the 8-puzzle, a state is the position of each of the eight tiles and the blank: a 3 × 3 array.
- In the water-jug problem, a state is an ordered pair (x, y) of the amounts of water in the two jugs.
- In route finding, a state is simply the name of the current city.

An **operator** (action) transforms one state into another. Each operator has:

- **Preconditions** — conditions that must hold in the current state for the operator to be applicable. "Fill the 4-litre jug" requires that it is not already full.
- **Effects** — how the state changes when the operator is applied. After filling, the 4-litre jug contains 4 litres.

Production Systems

Rich and Knight describe problem solving as search in a **production system**, which consists of:

1. A set of **production rules**, each of the form *condition* → *action*. The left side determines applicability; the right side describes the operation performed.
2. One or more **knowledge bases or databases** holding the information appropriate to the task (including the current state).
3. A **control strategy** that specifies the order in which rules are compared with the database and how to resolve conflicts when several rules match at once.
4. A **rule applier** that applies the selected rule.

A good control strategy must **cause motion** (it must change the state; a strategy that keeps applying a rule that does nothing never reaches a solution) and it must be **systematic** (it should not generate the same state again and again and should explore the space in an organised way). Breadth-first and depth-first search are the two basic systematic control strategies. The water-jug problem in the next chapter is the classic example of a production system.

State Space

The **state space** is the set of all states reachable from the initial state by any sequence of actions. It forms a directed graph — the **state-space graph** — in which the nodes are states and the arcs (links) between nodes are actions. If actions are reversible (as in the 8-puzzle or route finding), each arc can be drawn as two-way.

Solving a problem is then the same as finding a path in this graph from the initial state to a goal state. Search algorithms do this by building a **search tree**:

- The root of the tree is the initial state.
- **Expanding** a node means applying every applicable action to its state, **generating** a set of child nodes.
- The set of all leaf nodes available for expansion at any moment is called the **frontier** (also the **open list** or **fringe**).
- Nodes that have already been expanded may be kept in an **explored set** (also the **closed list**) to avoid repeated states.

It is important to distinguish **states** from **nodes**. A state is a configuration of the world. A node is a bookkeeping data structure in the search tree that contains a state, a pointer to its **parent** node, the **action** that generated it, the **path cost** g(n) from the root, and its **depth**. The same state may appear in many nodes if it can be reached by several paths. A state-space graph with loops (for example Arad → Sibiu → Arad) produces an infinite search tree; this is why the explored set, or a check for repeated states, is so important.

```
function TREE-SEARCH(problem) returns a solution or failure
  initialise the frontier using the initial state of problem
  loop do
    if the frontier is empty then return failure
    choose a leaf node and remove it from the frontier
    if the node contains a goal state then return the solution
    expand the node, adding the resulting nodes to the frontier

function GRAPH-SEARCH(problem) returns a solution or failure
  initialise the frontier using the initial state of problem
  initialise the explored set to be empty
  loop do
    if the frontier is empty then return failure
    choose a leaf node and remove it from the frontier
    if the node contains a goal state then return the solution
    add the node's state to the explored set
    expand the node, adding resulting nodes to the frontier
        only if their state is not in the frontier or explored set
```

The only difference between search strategies is **how they choose which frontier node to expand next**. That single choice gives breadth-first, depth-first, uniform-cost and all the other strategies of the next-but-one chapter.

> **Memory Trick:** "A node is a state in a backpack" — the node carries the state plus its luggage: parent, action, path cost and depth.

Problem Types

Problems differ according to what the agent knows about its state and about the effects of its actions. Russell and Norvig distinguish four types, illustrated with the two-square vacuum world (squares A and B, each possibly dirty; actions Left, Right, Suck).

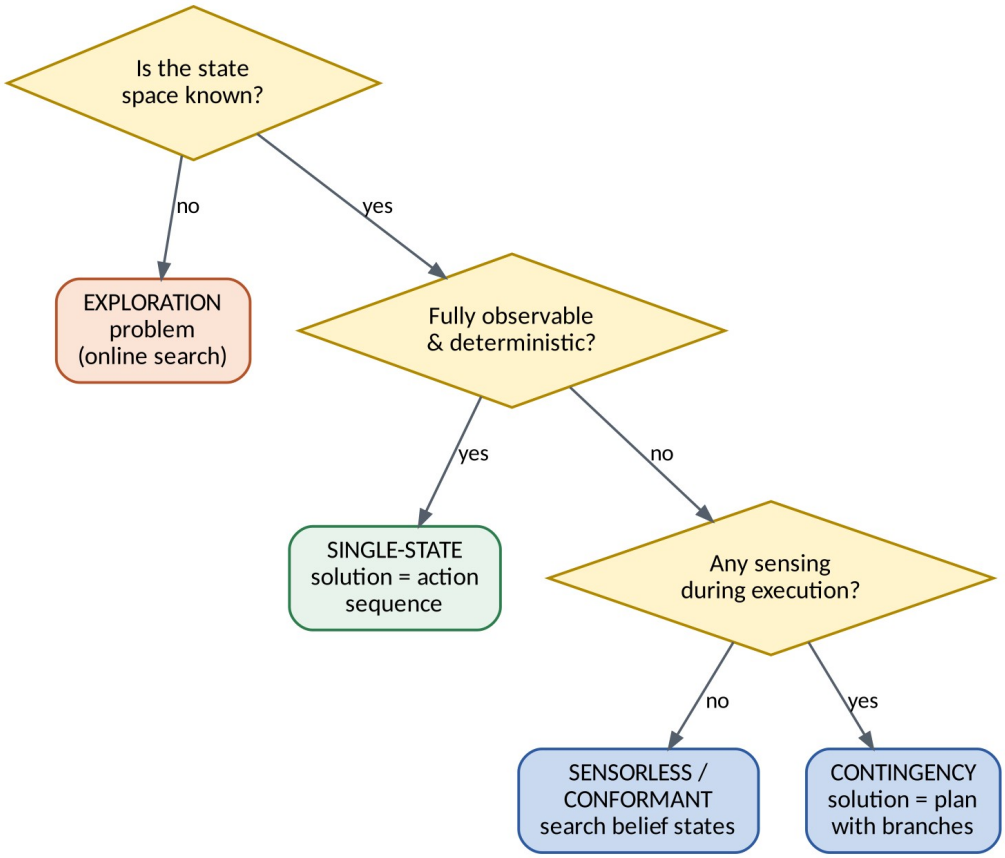


Figure 1.8: Deciding the problem type

Single-State Problems

The environment is **fully observable** and **deterministic**. The agent knows exactly which state it is in and exactly what each action will do. The solution is a fixed **sequence** of actions. Example: the vacuum agent knows it is in A and both squares are dirty; the solution is [Suck, Right, Suck].

Sensorless (Conformant) Problems

The agent has **no sensors** at all (or its percepts provide no information), but the actions are deterministic. It does not know which state it is in; it knows only that it is in one of a set of possible states, called a **belief state**. It must find a sequence of actions that reaches a goal whatever the actual state was. Example: the vacuum agent could be in any of the eight states, yet the sequence [Right, Suck, Left, Suck] is guaranteed to clean both squares, as worked out in the next chapter. The solution is still a sequence, but the search is through the space of belief states.

Contingency Problems

The environment is **partially observable** and/or **non-deterministic** (actions may have uncertain effects). The agent can perceive new information during execution. The solution is a **contingency plan** (a tree or policy with branches such as "if dirty then Suck"), and the agent often **interleaves** search and execution. Example: in the "erratic" vacuum world, sucking a dirty square sometimes also deposits dirt on a clean square, so the plan must check the dirt sensor after acting. If uncertainty comes from other agents, the problem is called **adversarial** (as in games).

Exploration Problems

The **state space is unknown**: the agent does not know which states exist or what its actions do. It must act in order to discover them — like a newborn baby or a robot placed in an unmapped building. This is the extreme case of a contingency problem and requires **online search**.

Table 1.15: The four problem types

Problem type	What the agent knows	Environment	Form of solution	Vacuum-world example
Single-state	Exact current state; exact action effects	Fully observable, deterministic	Action sequence	Start in A with both dirty: [Suck, Right, Suck]
Sensorless (conformant)	Only a set of possible states (belief state)	Unobservable, deterministic	Action sequence that works for all states	[Right, Suck, Left, Suck] from any state
Contingency	Partial state; outcomes may vary	Partially observable and/or non-deterministic	Conditional plan or policy; interleave search and action	Erratic sucking: [Suck, if dirty then Suck …]
Exploration	Nothing about states or effects	Unknown state space	Learn while acting (online search)	Robot in an unmapped house

Toy Problems and Real-World Problems

Problems used in AI are also classified by purpose.

- A **toy problem** is intended to illustrate or exercise problem-solving methods. It has a concise, exact description, so different researchers can compare the performance of algorithms on it. Examples: vacuum world, 8-puzzle, 8-queens, water jug, missionaries and cannibals, Tower of Hanoi, crypt-arithmetic.
- A **real-world problem** is one whose solutions people actually care about. Such problems tend not to have a single agreed description. Examples: route finding, touring problems and the travelling salesman problem, VLSI layout, robot navigation, automatic assembly sequencing, airline travel planning, protein design.

Table 1.16: Toy problems versus real-world problems

Aspect	Toy problem	Real-world problem
Purpose	Illustrate and compare algorithms	Solve a problem people care about
Description	Concise, exact, universally agreed	Messy; many possible formulations
Size	Small to moderate state space	Often huge or continuous
Examples	8-puzzle, 8-queens, water jug, vacuum world, M and C	Route finding, TSP, VLSI layout, robot navigation

Problem Characteristics (Rich and Knight)

Before choosing a method it helps to analyse a problem along several dimensions:

Table 1.17: Problem characteristics used to choose a solution method

Question	Example answer
Is the problem decomposable into independent sub-problems?	Symbolic integration: yes; 8-puzzle: no

Question	Example answer
Can solution steps be ignored or undone?	Theorem proving: ignorable; 8-puzzle: recoverable; chess: irrecoverable
Is the universe predictable?	8-puzzle: certain outcome; bridge (cards): uncertain
Is a good solution absolute or relative?	Any path vs the best path (TSP needs the best)
Is the solution a state or a path?	8-queens: final state; water jug: path
What is the role of knowledge?	Chess needs little to check moves, much to play well
Does the task require interaction with a person?	Solitary vs conversational problem solving

Exam Tip: A favourite 5-mark question is "Explain the different problem types with examples." Use the vacuum world for all four types — it shows that the same physical world can produce different problem types depending on what the agent can sense and on whether its actions are reliable.

Summary

- An agent perceives through sensors and acts through actuators; a rational agent maximises expected performance. Task environments are described by PEAS and classified by observability, agents, determinism, episodes, dynamics, discreteness and knowledge.
- A problem-solving agent is goal-based: it formulates a goal, formulates a problem, searches for a solution and executes it.
- A problem has five components: initial state, actions (operators), transition model, goal test and path cost. The first three define the state space.
- States describe configurations; operators have preconditions and effects; production systems consist of rules, a database, a control strategy and a rule applier.
- The state space is a graph; search builds a tree of nodes (state + parent + action + path cost + depth) and differs only in which frontier node it expands next.
- Problem types: single-state, sensorless (conformant), contingency and exploration. Problems may also be toy or real-world.

Classic Problems: States, Operators and State Spaces

This chapter applies the problem-formulation method of the previous chapter to the classic problems that appear again and again in exams. For each one we define the states, initial state, operators (actions), transition model, goal test and path cost, and then look at the state space and a solution. Practise writing these formulations from memory — they are the raw material for every search question.

The Vacuum World

The vacuum world is the simplest toy problem. A robot vacuum cleaner lives in a world of two squares, A (left) and B (right). Each square may or may not contain dirt.

- **States:** The state is determined by the agent's location and the dirt status of each square. The agent is in one of 2 squares, and each square is dirty or clean, so there are 2 × 2 × 2 = 8 possible states. (A larger world with n squares has n × 2^n states.)
- **Initial state:** Any state can be designated as the initial state.
- **Actions:** Left, Right and Suck. (Larger versions add NoOp.)
- **Transition model:** Left moves the agent to A, Right moves it to B, Suck removes dirt from the current square. Moving Left in A, moving Right in B, and sucking a clean square have no effect.
- **Goal test:** Both squares are clean.
- **Path cost:** Each step costs 1, so path cost is the number of steps.

We number the states as follows: S1 = agent in A, both dirty; S2 = agent in B, both dirty; S3 = agent in A, only A dirty; S4 = agent in B, only A dirty; S5 = agent in A, only B dirty; S6 = agent in B, only B dirty; S7 = agent in A, both clean; S8 = agent in B, both clean.

Table 1.18: Transition table of the vacuum world

State	Left	Right	Suck	Goal?
S1 (A; A dirty, B dirty)	S1	S2	S5	No
S2 (B; A dirty, B dirty)	S1	S2	S4	No
S3 (A; A dirty, B clean)	S3	S4	S7	No
S4 (B; A dirty, B clean)	S3	S4	S4	No
S5 (A; A clean, B dirty)	S5	S6	S5	No
S6 (B; A clean, B dirty)	S5	S6	S8	No
S7 (A; both clean)	S7	S8	S7	Yes
S8 (B; both clean)	S7	S8	S8	Yes

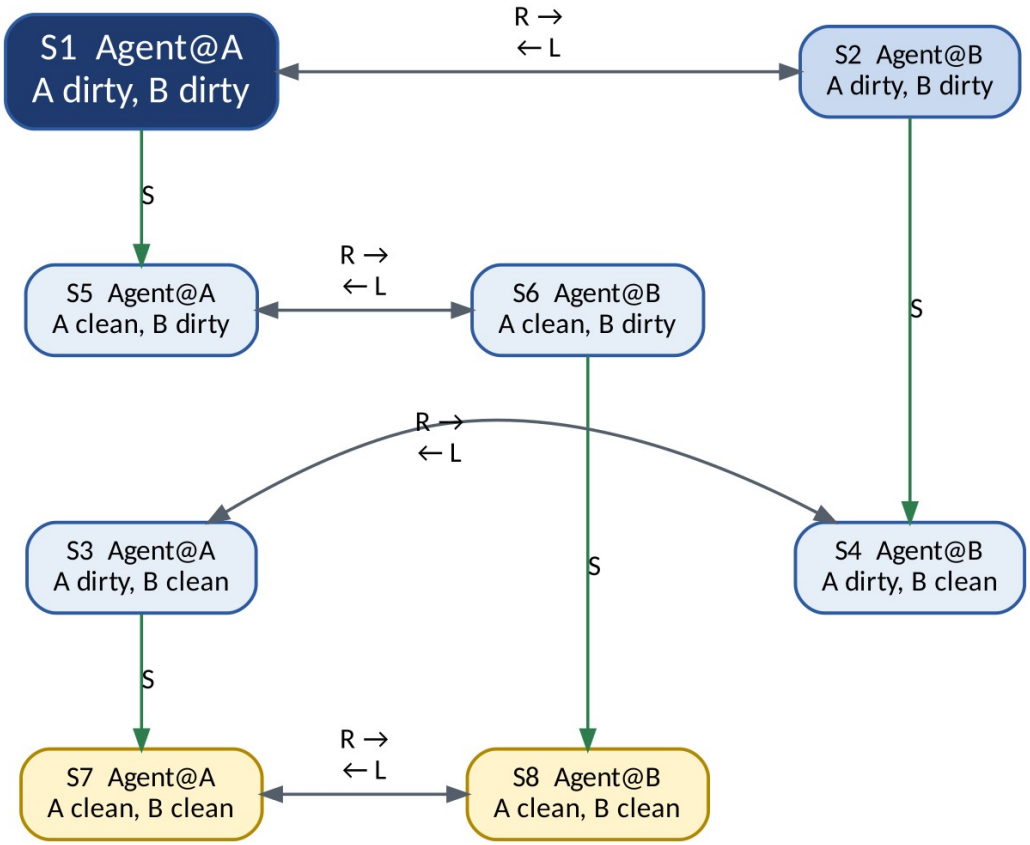


Figure 1.9: State space of the vacuum world (self-loops omitted; S = Suck)

From S1, the optimal solution is [Suck, Right, Suck]: S1 → S5 → S6 → S8, with path cost 3.

Worked Example — Sensorless vacuum world: Suppose the agent has no sensors, so it believes it may be in any state: belief state {S1, ..., S8}. Apply Right: every state moves the agent to B, giving {S2, S4, S6, S8}. Apply Suck: B becomes clean in each, giving {S4, S8}. Apply Left: {S3, S7}. Apply Suck: A becomes clean, giving {S7}. The sequence [Right, Suck, Left, Suck] therefore reaches a goal whatever the starting state was — a conformant solution found by searching in the space of belief states.

The 8-Puzzle

The 8-puzzle consists of a 3 × 3 board with eight numbered tiles and one blank space. A tile adjacent to the blank can slide into the blank. The objective is to reach a specified goal configuration. It belongs to the family of **sliding-block puzzles**, which are often used as test problems for new search algorithms.

- **States:** A state specifies the location of each of the eight tiles and the blank in one of the nine squares.
- **Initial state:** Any state. (Exactly half of all arrangements can reach any given goal.)
- **Actions:** It is simplest to think of the **blank** moving: Left, Right, Up or Down. Different subsets are possible depending on where the blank is (2 moves in a corner, 3 on an edge, 4 in the centre).
- **Transition model:** Given a state and an action, returns the resulting state — the blank swaps with the neighbouring tile in that direction.
- **Goal test:** Checks whether the state matches the goal configuration.
- **Path cost:** Each step costs 1, so the path cost is the number of moves.

Table 1.19: Sizes of sliding-tile puzzle state spaces

Property	8-puzzle (3 × 3)	15-puzzle (4 × 4)	24-puzzle (5 × 5)
Reachable states	9!/2 = 181,440	16!/2 ≈ 1.05 × 10^13	25!/2 ≈ 7.8 × 10^24
Branching factor	2 to 4 (about 3 on average)	2 to 4	2 to 4
Practical solution	Easily solved optimally	Solved optimally in milliseconds with good heuristics	Hard; needs strong heuristics

The 8-puzzle is **NP-complete** in its general n × n form, so no known algorithm solves large instances efficiently in the worst case.

Solving an 8-puzzle instance. Initial state (rows, _ is the blank): 2 8 3 / 1 6 4 / 7 _ 5. Goal: 1 2 3 / 8 _ 4 / 7 6 5. Moving the blank Up gives 2 8 3 / 1 _ 4 / 7 6 5. Up again gives 2 _ 3 / 1 8 4 / 7 6 5. Left gives _ 2 3 / 1 8 4 / 7 6 5. Down gives 1 2 3 / _ 8 4 / 7 6 5. Right gives 1 2 3 / 8 _ 4 / 7 6 5, the goal. The solution [Up, Up, Left, Down, Right] has cost 5, and a breadth-first search confirms that no shorter solution exists.

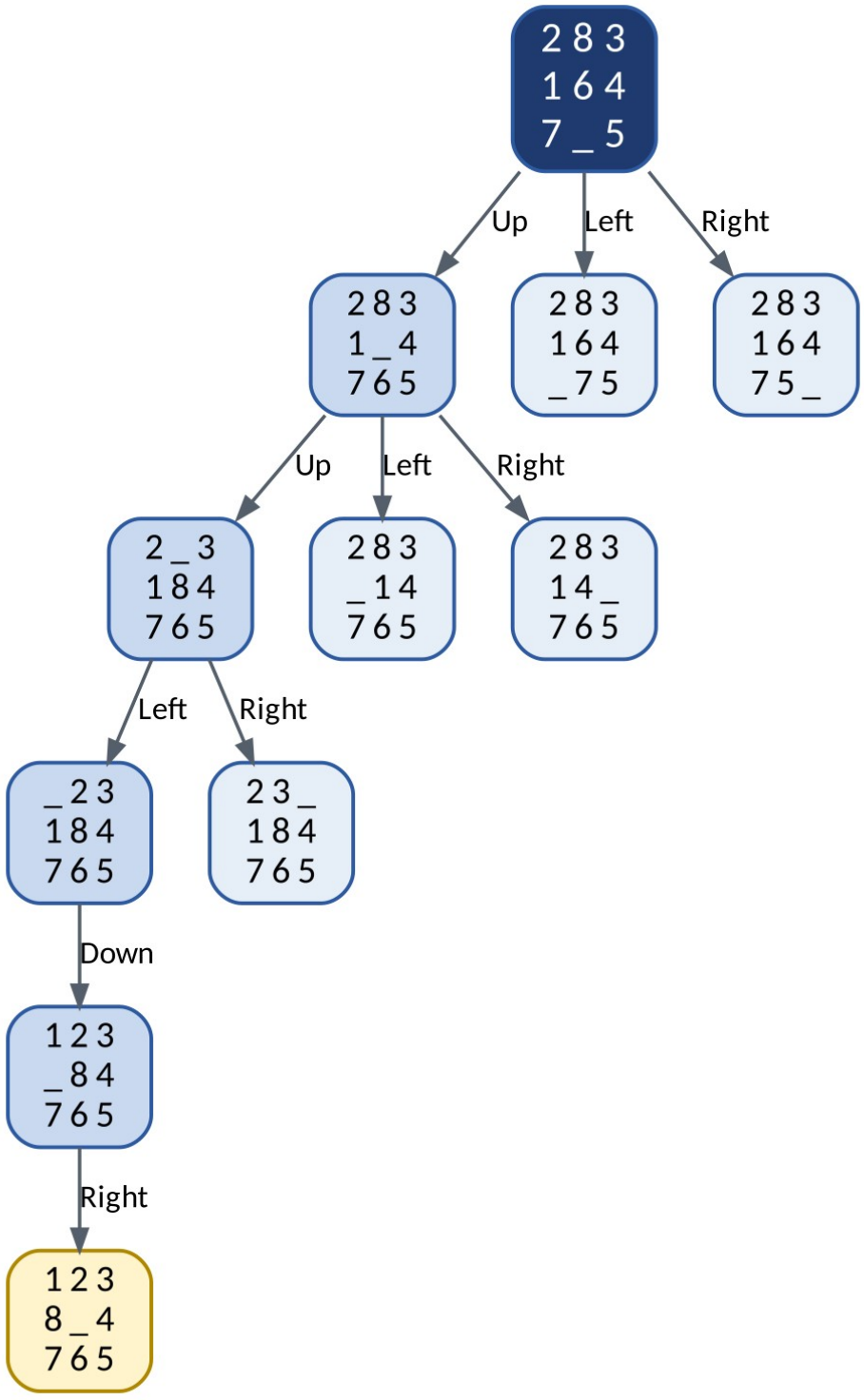


Figure 1.10: Fragment of the 8-puzzle state space for the worked example (solution path shaded)

Notice that in the figure the move that would undo the previous move (for example, Down straight after Up) is not drawn. A search algorithm that does not check for repeated states would generate these "undo" moves and waste effort — the 8-puzzle state-space graph is full of cycles.

The 8-Queens Problem

The goal of the 8-queens problem is to place eight queens on a chessboard so that no queen attacks any other. A queen attacks any piece in the same row, column or diagonal. Although efficient special-purpose algorithms exist for the whole n-queens family, it remains a useful test problem for search algorithms.

There are two main kinds of formulation. An **incremental formulation** starts with an empty board and adds a queen at each step. A **complete-state formulation** starts with all eight queens on the board and moves them around.

Incremental Formulation (Naive)

- **States:** Any arrangement of 0 to 8 queens on the board.
- **Initial state:** No queens on the board.
- **Actions:** Add a queen to any empty square.
- **Transition model:** Returns the board with the queen added.
- **Goal test:** 8 queens are on the board, none attacked.
- **Path cost:** Not relevant — only the final state matters.

In this formulation there are 64 × 63 × … × 57 ≈ 1.8 × 10^14 possible sequences to investigate.

Improved Incremental Formulation

- **States:** All arrangements of n queens (0 ≤ n ≤ 8), one per column in the leftmost n columns, with no queen attacking another.
- **Actions:** Add a queen to any square in the leftmost empty column such that it is not attacked by any other queen.

This reduces the state space from 1.8 × 10^14 to just **2,057** states, and solutions are easy to find. The lesson: a good formulation can shrink a search space dramatically.

Complete-State Formulation

- **States:** Eight queens on the board, one per column (8^8 = 16,777,216 states).
- **Actions:** Move a queen to another square in the same column.
- **Goal test:** No pair of queens attacks each other.

This formulation is used by local search methods such as hill climbing, described in the next chapter, where the number of attacking pairs serves as the value to minimise.

Table 1.20: Three formulations of the 8-queens problem

Formulation	States	State-space size	Typical algorithm
Naive incremental	0–8 queens anywhere	≈ 1.8 × 10^14 sequences	Not practical
Improved incremental	Non-attacking queens in leftmost columns	2,057	DFS / backtracking
Complete-state	8 queens, one per column	8^8 ≈ 1.7 × 10^7	Hill climbing, local search

The 8-queens problem has **92** solutions (12 if rotations and reflections are treated as the same). One solution, giving the row of the queen in columns 1 to 8, is (1, 5, 8, 6, 3, 7, 2, 4). Check: all rows are different, and for every pair of columns i, j the row difference is never equal to the column difference, so no two queens share a diagonal. Note that 8-queens is a problem where the **solution is a state**, not a path.

The Water-Jug Problem

This is the classic production-system problem from Rich and Knight, and probably the single most asked problem in this unit.

Problem statement. You are given two jugs, a 4-litre one and a 3-litre one. Neither has any measuring marks. There is a pump (tap) that can be used to fill the jugs with water, and water can be poured away on the ground. How can you get exactly 2 litres of water into the 4-litre jug?

Formulation

- **States:** An ordered pair (x, y), where x = litres in the 4-litre jug (x = 0, 1, 2, 3, 4) and y = litres in the 3-litre jug (y = 0, 1, 2, 3). There are 5 × 4 = 20 possible pairs, of which 14 are actually reachable from (0, 0).
- **Initial state:** (0, 0) — both jugs empty.
- **Goal state:** (2, n) for any n — 2 litres in the 4-litre jug.
- **Operators:** The production rules below.
- **Path cost:** 1 per operation (each fill, empty or pour counts once).

Production Rules

Table 1.21: Production rules for the water-jug problem

Rule	Current state and condition	New state	Meaning
R1	(x, y), if x < 4	(4, y)	Fill the 4-litre jug
R2	(x, y), if y < 3	(x, 3)	Fill the 3-litre jug
R3	(x, y), if x > 0	(x − d, y)	Pour some water out of the 4-litre jug
R4	(x, y), if y > 0	(x, y − d)	Pour some water out of the 3-litre jug
R5	(x, y), if x > 0	(0, y)	Empty the 4-litre jug on the ground
R6	(x, y), if y > 0	(x, 0)	Empty the 3-litre jug on the ground
R7	(x, y), if x + y ≥ 4 and y > 0	(4, y − (4 − x))	Pour from 3-litre into 4-litre until 4-litre is full
R8	(x, y), if x + y ≥ 3 and x > 0	(x − (3 − y), 3)	Pour from 4-litre into 3-litre until 3-litre is full
R9	(x, y), if x + y ≤ 4 and y > 0	(x + y, 0)	Pour all water from 3-litre into 4-litre
R10	(x, y), if x + y ≤ 3 and x > 0	(0, x + y)	Pour all water from 4-litre into 3-litre
R11	(0, 2)	(2, 0)	Pour the 2 litres from 3-litre into 4-litre
R12	(2, y)	(0, y)	Empty the 2 litres in the 4-litre jug on the ground

Rules R3 and R4 ("pour some water out") are legal but useless here, because the jugs have no marks — you cannot pour out a measured amount d. Rules R11 and R12 are special cases of R9 and R5 included by Rich and Knight to show that a rule set may contain redundant, more specific rules; a control strategy may prefer them.

Solution 1 (Starting by Filling the 3-Litre Jug)

Table 1.22: Water-jug solution path 1 (six steps)

Step	4-litre jug (x)	3-litre jug (y)	Rule applied	Action
0	0	0	—	Start: both empty
1	0	3	R2	Fill the 3-litre jug
2	3	0	R9	Pour all of 3-litre into 4-litre
3	3	3	R2	Fill the 3-litre jug again
4	4	2	R7	Pour from 3-litre into 4-litre until full (1 litre moves)
5	0	2	R5	Empty the 4-litre jug
6	2	0	R9 (or R11)	Pour the 2 litres into the 4-litre jug — goal

Solution 2 (Starting by Filling the 4-Litre Jug)

Table 1.23: Water-jug solution path 2 (six steps)

Step	4-litre jug (x)	3-litre jug (y)	Rule applied	Action
0	0	0	—	Start: both empty
1	4	0	R1	Fill the 4-litre jug
2	1	3	R8	Pour from 4-litre into 3-litre until full
3	1	0	R6	Empty the 3-litre jug
4	0	1	R10	Pour all of 4-litre into 3-litre
5	4	1	R1	Fill the 4-litre jug
6	2	3	R8	Pour from 4-litre into 3-litre until full (2 litres move) — goal

Both solutions take six operations. A breadth-first search over the 14 reachable states confirms that six is the minimum.

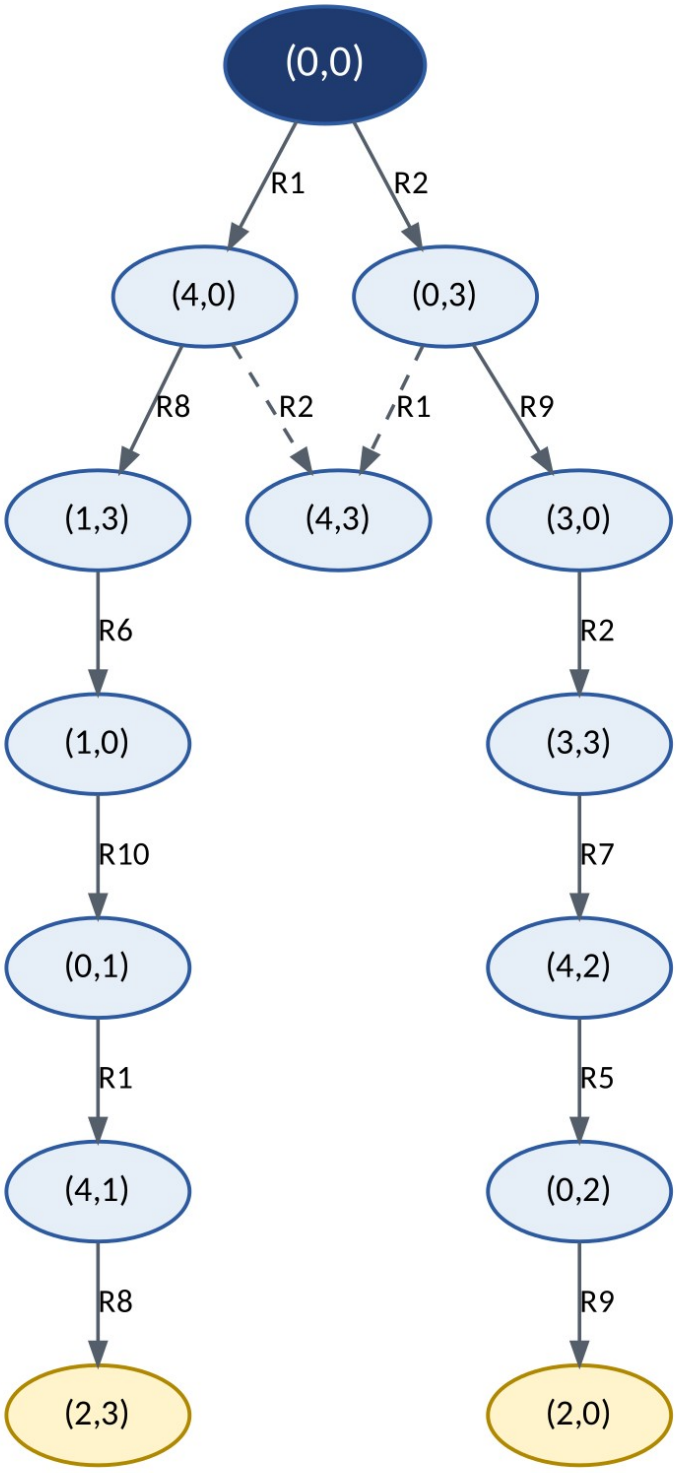


Figure 1.11: Partial state space of the water-jug problem showing both six-step solutions

Worked Example — Breadth-first search on the water-jug problem: Level 0: (0,0). Level 1 (apply all rules): (4,0) by R1 and (0,3) by R2. Level 2: from (4,0) we get (4,3) by R2 and (1,3) by R8; from (0,3) we get (3,0) by R9 ((4,3) already seen). Level 3: from (1,3) we get (1,0) by R6; from (3,0) we get (3,3) by R2. Level 4: (0,1) by R10 and (4,2) by R7. Level 5: (4,1) by R1 and (0,2) by R5. Level 6: (2,3) by R8 from (4,1) — the goal test x = 2 succeeds. BFS therefore returns the path in Table 1.23, of length six, and it is guaranteed to be a shortest solution.

Exam Tip: In a 10-mark water-jug answer, write: (1) problem statement, (2) state representation (x, y) with ranges, (3) initial and goal states, (4) the full production-rule table, (5) at least one solution path as a table with rule numbers, (6) the state-space diagram, and (7) a line on the control strategy (BFS finds the shortest path). Examiners sometimes change the jug sizes (for example 5 L and 3 L, measure 4 L) — the method is identical, only the numbers in the rules change.

Missionaries and Cannibals

Problem statement. Three missionaries and three cannibals are on the left bank of a river, along with a boat that can hold one or two people. Find a way to get everyone to the right bank, without ever leaving a group of missionaries in one place outnumbered by the cannibals in that place (otherwise the missionaries are eaten). The boat cannot cross empty.

Formulation

- **States:** A triple (M, C, B), where M = number of missionaries on the left bank (0–3), C = number of cannibals on the left bank (0–3), and B = 1 if the boat is on the left bank, 0 if on the right. The right bank then has 3 − M missionaries and 3 − C cannibals.
- **Initial state:** (3, 3, 1).
- **Goal state:** (0, 0, 0).
- **Actions:** The boat carries one of five loads across: 1M, 2M, 1C, 2C, or 1M + 1C. If the boat is on the left the numbers are subtracted from the left bank; if on the right they are added.
- **Constraint (legality test):** On each bank, missionaries must be either zero or at least as many as cannibals: (M = 0 or M ≥ C) and (3 − M = 0 or 3 − M ≥ 3 − C).
- **Path cost:** 1 per crossing.

Of the 32 possible triples, 20 satisfy the safety constraint and 16 are reachable from the initial state.

Solution (11 Crossings)

Table 1.24: Missionaries and cannibals — an optimal 11-crossing solution

Step	Boat carries	Direction	Left bank (M, C)	Right bank (M, C)	State (M, C, B)
0	—	—	3, 3	0, 0	(3, 3, 1)
1	2 C	Left → Right	3, 1	0, 2	(3, 1, 0)
2	1 C	Right → Left	3, 2	0, 1	(3, 2, 1)
3	2 C	Left → Right	3, 0	0, 3	(3, 0, 0)
4	1 C	Right → Left	3, 1	0, 2	(3, 1, 1)
5	2 M	Left → Right	1, 1	2, 2	(1, 1, 0)
6	1 M + 1 C	Right → Left	2, 2	1, 1	(2, 2, 1)
7	2 M	Left → Right	0, 2	3, 1	(0, 2, 0)
8	1 C	Right → Left	0, 3	3, 0	(0, 3, 1)
9	2 C	Left → Right	0, 1	3, 2	(0, 1, 0)
10	1 C	Right → Left	0, 2	3, 1	(0, 2, 1)
11	2 C	Left → Right	0, 0	3, 3	(0, 0, 0) — goal

Check any row: for example, after step 6 the left bank has 2 M and 2 C (equal — safe) and the right bank has 1 M and 1 C (safe). After step 7, the left bank has no missionaries, so the two cannibals there harm no one. A breadth-first search confirms that 11 crossings is the minimum.

> **Key Point:** The hard part of missionaries and cannibals is step 6, where a missionary and a cannibal must come back together. Without systematic search most people get stuck there, because it looks like moving "backwards". This shows why systematic search strategies are valuable — they do not reject a move just because it seems to lose progress.

Route Finding: The Romania Map

Route finding is the standard real-world problem. We use a subset of the Romania road map from Russell and Norvig, with road distances in kilometres.

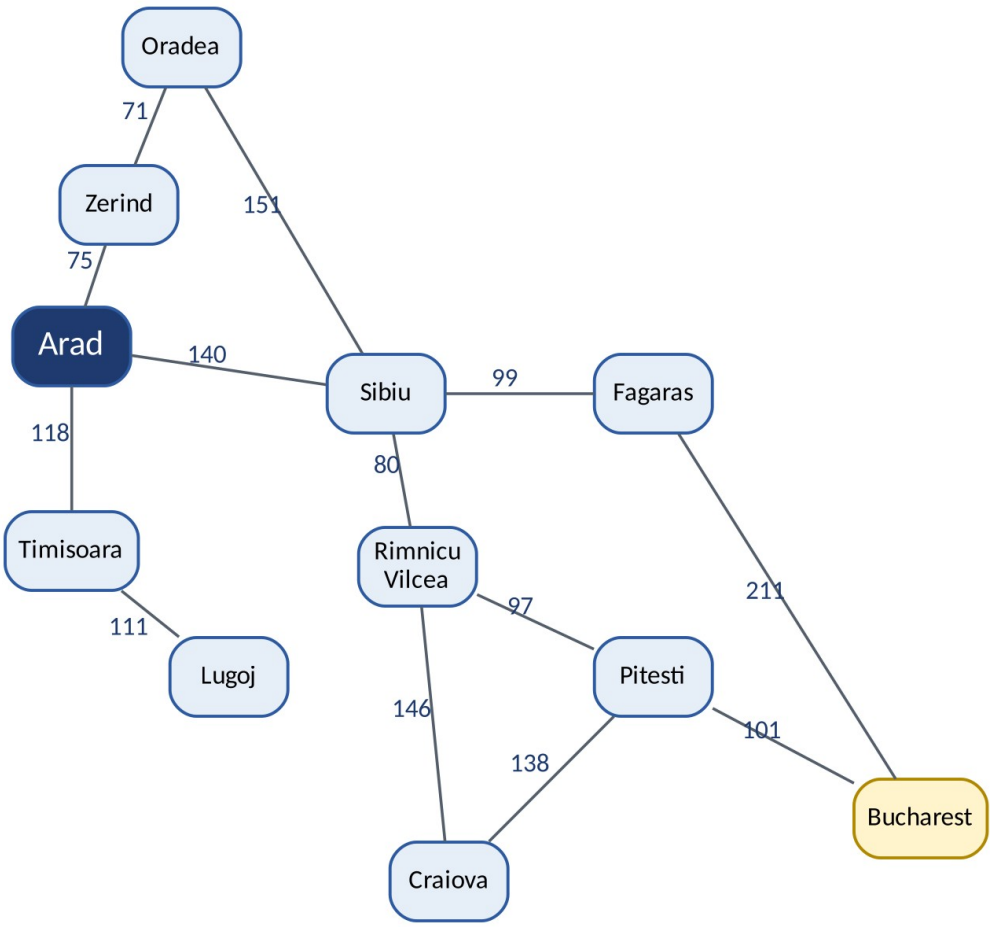


Figure 1.12: Subset of the Romania road map with road distances (km)

- **States:** Each state is a city: In(Arad), In(Sibiu), and so on.
- **Initial state:** In(Arad).
- **Actions:** Go(c) for each city c directly connected to the current city by a road. ACTIONS(In(Arad)) = {Go(Zerind), Go(Sibiu), Go(Timisoara)}.
- **Transition model:** RESULT(In(x), Go(y)) = In(y).
- **Goal test:** Is the state In(Bucharest)?
- **Path cost:** Sum of road distances along the route.

Some routes from Arad to Bucharest:

Table 1.25: Some routes from Arad to Bucharest

Route	Calculation	Cost (km)	Steps
Arad → Sibiu → Fagaras → Bucharest	140 + 99 + 211	450	3
Arad → Sibiu → Rimnicu Vilcea → Pitesti → Bucharest	140 + 80 + 97 + 101	418	4
Arad → Sibiu → Rimnicu Vilcea → Craiova → Pitesti → Bucharest	140 + 80 + 146 + 138 + 101	605	5

The route with the fewest steps (450 km) is not the cheapest (418 km). This distinction is crucial in the next chapter: breadth-first search finds the shallowest solution, while uniform-cost search finds the cheapest one.

Real route-finding systems include web-based driving directions, routing of video streams in computer networks, military operations planning and airline travel planning. Airline planning, for instance, must represent states by location *and* current time, and must include fare bases and connection times — a far richer formulation than the toy map.

Touring Problems and the Travelling Salesman Problem

Touring problems are related to route finding but the agent must visit every city at least once. The state therefore records not just the current city but also the **set of cities visited** so far. For example, the initial state might be In(Bucharest), Visited({Bucharest}).

The **travelling salesman problem (TSP)** is a touring problem in which each city must be visited **exactly once** and the agent returns to the start; the aim is to find the **shortest tour**. TSP is NP-hard. Apart from planning trips for salespeople, TSP algorithms are used for planning movements of automatic circuit-board drills and stocking machines on shop floors.

- **States:** (current city, set of visited cities).
- **Initial state:** (start city, {start city}).
- **Actions:** Go to any unvisited city; when all are visited, return to start.
- **Goal test:** All cities visited and agent back at the start.
- **Path cost:** Total distance.

For n cities there are (n − 1)!/2 distinct tours (fixing the start and treating a tour and its reverse as the same). For 10 cities this is 181,440; for 20 cities it is about 6 × 10^16, so brute-force enumeration quickly becomes impossible.

Example: a four-city TSP. Cities A, B, C, D with distances AB = 10, AC = 15, AD = 20, BC = 35, BD = 25, CD = 30. Fixing A as start there are (4 − 1)!/2 = 3 distinct tours. Tour A-B-C-D-A = 10 + 35 + 30 + 20 = 95. Tour A-B-D-C-A = 10 + 25 + 30 + 15 = 80. Tour A-C-B-D-A = 15 + 35 + 25 + 20 = 95. The optimal tour is A-B-D-C-A with length 80. This exhaustive method is an example of generate-and-test, discussed in the next chapter.

Other Real-World Problems

Table 1.26: Some real-world search problems

Problem	States	Actions	Goal / cost
VLSI layout	Positions of millions of components and wires on a chip	Place cell, route wire	Minimise area, delays and stray capacitance
Robot navigation	Continuous positions (and arm angles)	Continuous motions	Reach target; minimise time/energy
Automatic assembly sequencing	Partially assembled object	Add a part	Assembled object; feasible order
Protein design	Amino-acid sequence	Change an amino acid	Folds into 3-D shape with desired properties
Internet searching	Web pages	Follow links	Find pages answering a query

Memory Trick: "V-8-8-W-M-R" for the six classic problems — Vacuum, 8-puzzle, 8-queens, Water jug, Missionaries and cannibals, Romania route. The first five are toy problems; Romania (with TSP) represents real-world problems.

Summary

- Every problem is formulated with states, initial state, actions/operators, transition model, goal test and path cost.
- Vacuum world: 8 states; optimal plan from S1 is [Suck, Right, Suck]; sensorless plan [Right, Suck, Left, Suck] works from any state.
- 8-puzzle: 9!/2 = 181,440 reachable states; blank moves Up/Down/Left/Right; cost 1 per move.
- 8-queens: naive incremental ≈ 1.8 × 10^14 sequences; improved incremental 2,057 states; 92 solutions; solution is a state, not a path.
- Water jug (4 L, 3 L, measure 2 L): state (x, y), 12 production rules, shortest solutions take six steps.
- Missionaries and cannibals: state (M, C, B); optimal solution 11 crossings.
- Route finding (Romania) and TSP are real-world problems; fewest steps is not always least cost.

Search Strategies

Having formulated a problem, the agent must solve it by **searching** the state space. A **search strategy** is the rule that decides which node on the frontier to expand next. This chapter covers how to measure a strategy, the six **uninformed (blind) search** strategies — breadth-first, uniform-cost, depth-first, depth-limited, iterative deepening and bidirectional search — with a hand trace of each, and then gives a short introduction to **informed (heuristic) search**, generate-and-test and hill climbing as a bridge to Unit 2.

Infrastructure for Search Algorithms

Each search algorithm builds a search tree whose nodes are data structures with four (or five) components:

- **n.STATE** — the state in the state space to which the node corresponds;
- **n.PARENT** — the node that generated this node;
- **n.ACTION** — the action applied to the parent to generate the node;
- **n.PATH-COST** — the cost g(n) of the path from the initial state to the node;
- **n.DEPTH** — the number of steps along the path from the root (often also stored).

Following the PARENT pointers from a goal node back to the root gives the solution path. The frontier is stored in a **queue**, and the type of queue largely determines the strategy:

- **FIFO queue** (first in, first out) — gives breadth-first search.
- **LIFO queue or stack** (last in, first out) — gives depth-first search.
- **Priority queue** (pop the element with the best priority value) — gives uniform-cost search, and greedy and A* search in Unit 2.

Measuring Problem-Solving Performance

Strategies are evaluated along four dimensions.

Table 1.27: Criteria for evaluating search strategies

Criterion	Question it answers
Completeness	Is the algorithm guaranteed to find a solution when there is one?
Optimality	Does it find the optimal (lowest path cost) solution?
Time complexity	How long does it take to find a solution (number of nodes generated)?
Space complexity	How much memory is needed (maximum number of nodes stored)?

In AI the state-space graph is usually represented implicitly by the initial state, actions and transition model, and is frequently infinite. So complexity is expressed in terms of three quantities:

- **b** — the **branching factor**, the maximum number of successors of any node;
- **d** — the **depth** of the shallowest goal node (the number of steps along the path from the root);
- **m** — the **maximum length** of any path in the state space (may be infinite).

Time is measured as the number of nodes generated during the search, and space as the maximum number of nodes stored in memory. The **search cost** usually means time complexity (sometimes also memory); the **total cost** combines the search cost with the path cost of the solution found.

To remember the criteria, think **COTS** — Completeness, Optimality, Time, Space. And "b-d-m": branching, depth of best goal, maximum depth. Breadth worries about d; depth-first worries about m.

Uninformed versus Informed Search

Uninformed search (also called **blind search**) strategies have no additional information about states beyond that provided in the problem definition. All they can do is generate successors and distinguish a goal state from a non-goal state. They differ only in the order in which nodes are expanded.

Informed search (or **heuristic search**) strategies know whether one non-goal state is "more promising" than another, using a problem-specific estimate of the distance to the goal. They are introduced at the end of this chapter and studied fully in Unit 2.

The Common Example Tree

To compare the strategies fairly, we trace each on the same tree (Figure 1.13). The root is A, the goal is K at depth 3, and children are generated in alphabetical (left-to-right) order. The branching factor is at most 2.

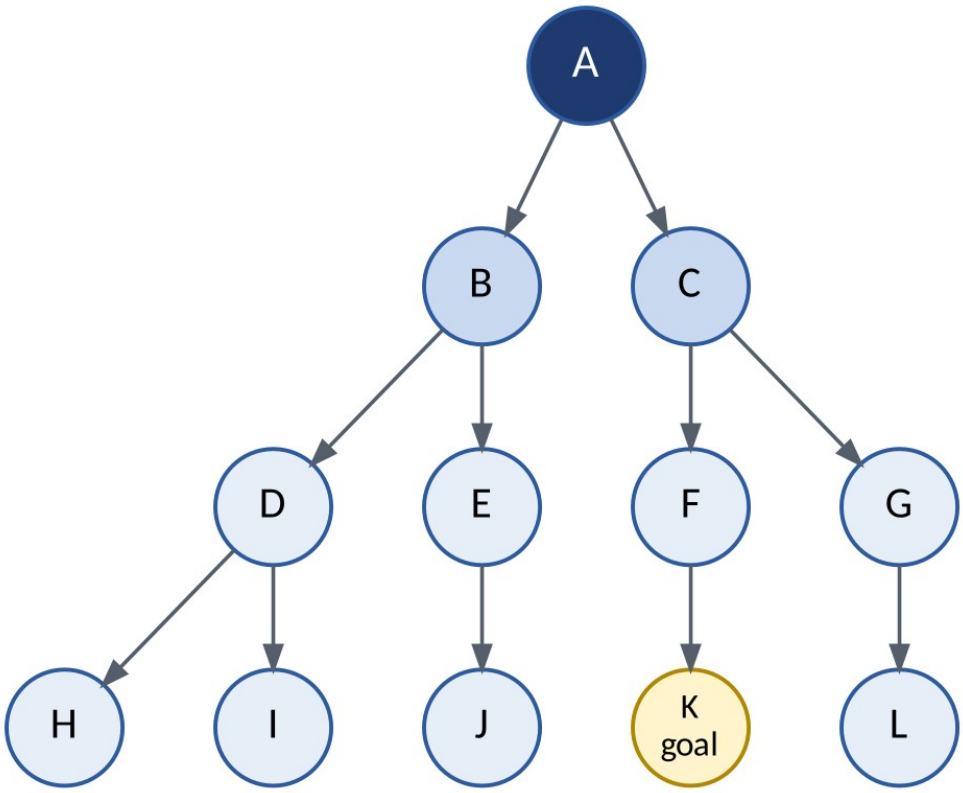


Figure 1.13: Example search tree used for the traces (goal K at depth 3)

The tree's edges are: A → B, C; B → D, E; C → F, G; D → H, I; E → J; F → K; G → L.

For the cost-sensitive strategy (uniform-cost search) and for bidirectional search we use the Romania map of Figure 1.12.

Breadth-First Search (BFS)

Breadth-first search expands the root node first, then all the successors of the root, then all their successors, and so on. All nodes at depth k are expanded before any node at depth k + 1. It uses a **FIFO**

queue for the frontier: new nodes (which are always deeper than their parents) go to the back of the queue, and old, shallower nodes are expanded first.

Algorithm

```
function BREADTH-FIRST-SEARCH(problem) returns a solution or failure
  node <- a node with STATE = problem.INITIAL-STATE, PATH-COST = 0
  if problem.GOAL-TEST(node.STATE) then return SOLUTION(node)
  frontier <- a FIFO queue with node as the only element
  explored <- an empty set
  loop do
    if EMPTY?(frontier) then return failure
    node <- POP(frontier)              /* shallowest node */
    add node.STATE to explored
    for each action in problem.ACTIONS(node.STATE) do
      child <- CHILD-NODE(problem, node, action)
      if child.STATE is not in explored or frontier then
        if problem.GOAL-TEST(child.STATE) then return SOLUTION(child)
        frontier <- INSERT(child, frontier)
```

Note that this version applies the goal test when a node is **generated**, which saves one whole level of work. Many textbooks (and many exam answers) apply the goal test when a node is **expanded** (popped) instead; this is simpler to trace and is what we do in the trace below.

Trace on the Example Tree

Table 1.28: BFS trace on the example tree (goal test on expansion)

Step	Node expanded	Frontier after expansion (front → back)
0	—	A
1	A	B, C
2	B	C, D, E
3	C	D, E, F, G
4	D	E, F, G, H, I
5	E	F, G, H, I, J
6	F	G, H, I, J, K
7	G	H, I, J, K, L
8	H	I, J, K, L
9	I	J, K, L
10	J	K, L
11	K — goal test succeeds	Stop

Order of expansion: **A, B, C, D, E, F, G, H, I, J, K**. Solution path: A → C → F → K. If the goal test is applied on generation, BFS stops at step 6, when K is generated as a child of F.

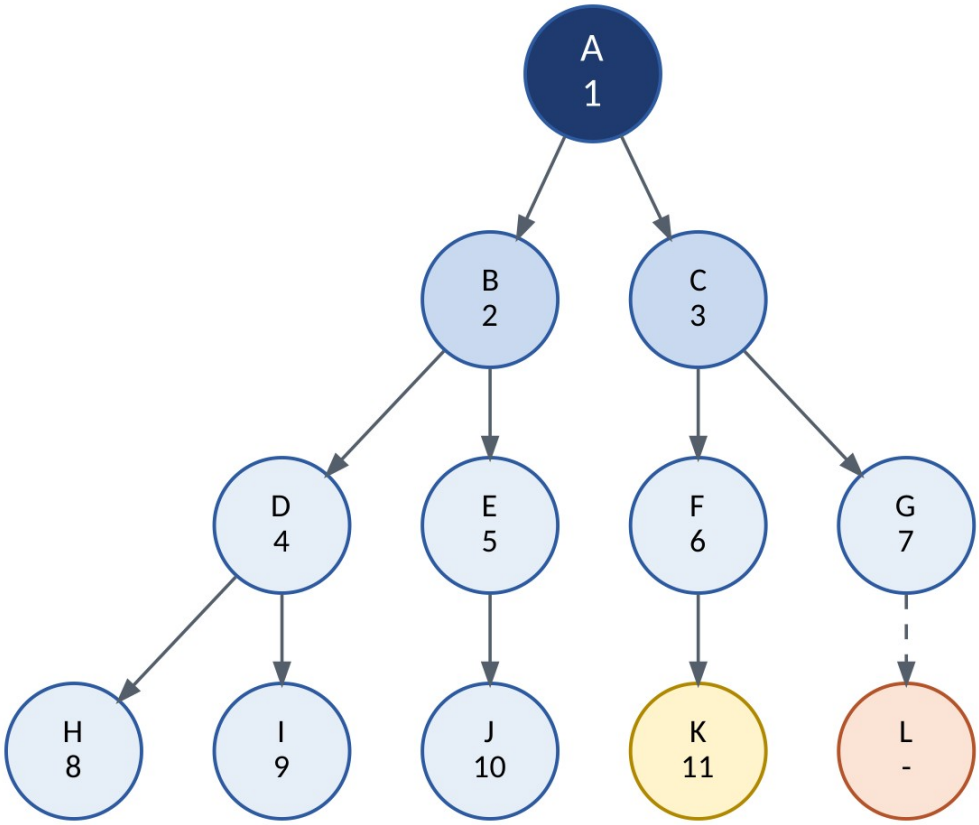


Figure 1.14: BFS expansion order on the example tree (numbers show the order; L is generated but never expanded)

Properties of BFS

- **Complete:** Yes, if b is finite — if the shallowest goal is at finite depth d, BFS will reach it after generating all shallower nodes.
- **Optimal:** Yes, if all step costs are equal (or path cost is a non-decreasing function of depth). It always finds the **shallowest** goal, which is not necessarily the cheapest in general.
- **Time:** b + b^2 + b^3 + … + b^d = O(b^d) with the goal test on generation; O(b^(d+1)) if the goal test is applied on expansion.
- **Space:** O(b^d) — every generated node stays in memory (O(b^(d−1)) in the explored set and O(b^d) in the frontier).

The Memory Problem of BFS

Exponential complexity is frightening. The table assumes b = 10, that one million nodes can be generated per second, and that a node needs 1,000 bytes of storage (assumptions used in the standard textbook).

Table 1.29: Time and memory requirements of BFS (b = 10)

Depth d	Nodes (about 1.11 × 10^d)	Time	Memory
2	111	0.11 milliseconds	111 kilobytes
4	11,111	11 milliseconds	11 megabytes
6	about 10^6	1.1 seconds	1.1 gigabytes
8	about 10^8	2 minutes	111 gigabytes
10	about 10^10	3 hours	11 terabytes

Depth d	Nodes (about 1.11 × 10^d)	Time	Memory
12	about 10^12	13 days	1.1 petabytes
14	about 10^14	3.5 years	111 petabytes

Two lessons follow. First, **memory is a bigger problem than time** for BFS: one might wait 13 days for a depth-12 answer, but no personal computer has a petabyte of memory. Second, exponential-complexity search problems cannot be solved by uninformed methods for any but the smallest instances.

Advantages of BFS: complete; finds the shallowest (minimum-step) solution; never gets trapped in an infinite branch. **Disadvantages:** exponential memory; slow when the goal is deep; not optimal for unequal step costs.

Uniform-Cost Search (UCS)

When step costs differ, BFS is not optimal. **Uniform-cost search** expands the node n with the **lowest path cost g(n)**. The frontier is a **priority queue ordered by g**. It is essentially Dijkstra's shortest-path algorithm applied to an implicit graph.

Two differences from BFS are important:

1. The **goal test is applied when a node is selected for expansion**, not when it is generated. The first goal node generated may be on a sub-optimal path.
2. If a better path is found to a node already on the frontier, the frontier entry is **replaced** by the cheaper one.

Algorithm

```
function UNIFORM-COST-SEARCH(problem) returns a solution or failure
  node <- a node with STATE = problem.INITIAL-STATE, PATH-COST = 0
  frontier <- a priority queue ordered by PATH-COST, with node
  explored <- an empty set
  loop do
    if EMPTY?(frontier) then return failure
    node <- POP(frontier)              /* lowest-cost node */
    if problem.GOAL-TEST(node.STATE) then return SOLUTION(node)
    add node.STATE to explored
    for each action in problem.ACTIONS(node.STATE) do
      child <- CHILD-NODE(problem, node, action)
      if child.STATE is not in explored or frontier then
        frontier <- INSERT(child, frontier)
      else if child.STATE is in frontier with higher PATH-COST then
        replace that frontier node with child
```

Trace: Arad to Bucharest

Using the map in Figure 1.12 (numbers in brackets are g values):

Table 1.30: Uniform-cost search trace from Arad to Bucharest

Step	Node expanded (g)	Frontier after expansion, sorted by g
1	Arad (0)	Zerind (75), Timisoara (118), Sibiu (140)
2	Zerind (75)	Timisoara (118), Sibiu (140), Oradea (146)
3	Timisoara (118)	Sibiu (140), Oradea (146), Lugoj (229)

Step	Node expanded (g)	Frontier after expansion, sorted by g
4	Sibiu (140)	Oradea (146), Rimnicu Vilcea (220), Lugoj (229), Fagaras (239)
5	Oradea (146)	Rimnicu Vilcea (220), Lugoj (229), Fagaras (239)
6	Rimnicu Vilcea (220)	Lugoj (229), Fagaras (239), Pitesti (317), Craiova (366)
7	Lugoj (229)	Fagaras (239), Pitesti (317), Craiova (366)
8	Fagaras (239)	Pitesti (317), Craiova (366), Bucharest (450)
9	Pitesti (317)	Craiova (366), Bucharest (418) — replaces 450
10	Craiova (366)	Bucharest (418)
11	Bucharest (418) — goal test succeeds	Stop

In step 5, Oradea's successor Sibiu (146 + 151 = 297) is ignored because Sibiu is already explored. In step 8, Bucharest is **generated** with cost 450 via Fagaras, but UCS does not stop: a goal test on generation would have returned the sub-optimal 450 km route. In step 9 a cheaper path to Bucharest (317 + 101 = 418) replaces the old one. The solution is **Arad → Sibiu → Rimnicu Vilcea → Pitesti → Bucharest, cost 418 km**, which is optimal.

Properties of UCS

- **Complete:** Yes, provided every step cost is at least some small positive constant ε (otherwise it can loop forever along zero-cost actions).
- **Optimal:** Yes — nodes are expanded in order of their optimal path cost, so the first goal expanded is the cheapest.
- **Time and space:** O(b^(1 + ⌊C*/ε⌋)), where C* is the cost of the optimal solution. This can be much greater than b^d, because UCS may explore large trees of small steps before exploring paths with large, useful steps.
- When all step costs are equal, b^(1 + ⌊C*/ε⌋) is just b^(d+1), and UCS behaves like BFS (except that it goes one level further because of the later goal test).

> **Key Point:** BFS minimises the number of steps; UCS minimises the total path cost. They give the same answer only when every step has the same cost. On the Romania map, BFS returns the 3-step, 450 km route via Fagaras while UCS returns the 4-step, 418 km route via Rimnicu Vilcea and Pitesti.

Depth-First Search (DFS)

Depth-first search always expands the **deepest** node in the current frontier. The search proceeds immediately to the deepest level of the tree, where nodes have no successors. As those nodes are expanded, they are dropped from the frontier, and the search "backs up" to the next deepest node that still has unexplored successors. DFS uses a **LIFO queue (stack)**; it is often implemented recursively.

Algorithm

```
function DEPTH-FIRST-SEARCH(problem) returns a solution or failure
  frontier <- a stack containing a node for problem.INITIAL-STATE
  loop do
```

```
    if EMPTY?(frontier) then return failure
    node <- POP(frontier)              /* most recently added */
    if problem.GOAL-TEST(node.STATE) then return SOLUTION(node)
    push the children of node onto frontier in reverse order,
        so that the leftmost child is popped first
        (graph-search version: skip states already explored)
```

Trace on the Example Tree

Table 1.31: DFS trace on the example tree

Step	Node popped	Stack after pushing children (top at right)
0	—	A
1	A	C, B
2	B	C, E, D
3	D	C, E, I, H
4	H (leaf)	C, E, I
5	I (leaf)	C, E
6	E	C, J
7	J (leaf)	C
8	C	G, F
9	F	G, K
10	K — goal test succeeds	Stop

Order of expansion: **A, B, D, H, I, E, J, C, F, K**. Solution path: A → C → F → K. Nodes G and L are never visited. Notice how small the stack stays: at most four nodes, compared with five or more in the BFS queue.

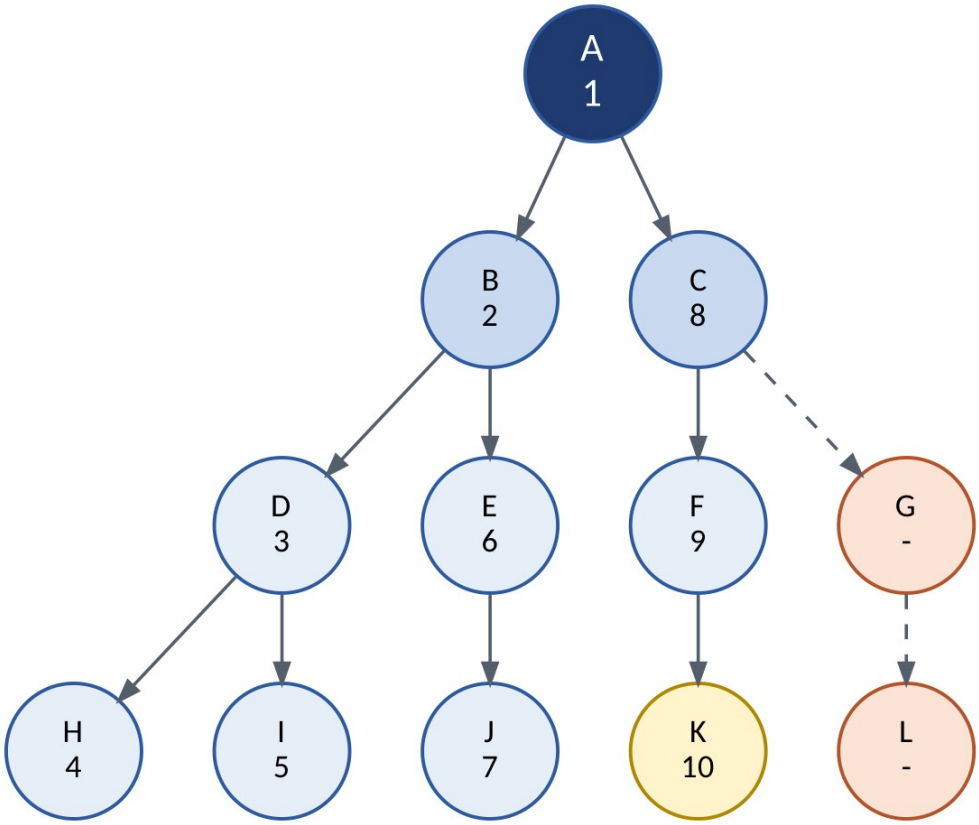


Figure 1.15: DFS expansion order on the example tree (G and L are never visited)

Properties of DFS

- **Complete:** The graph-search version is complete in **finite** state spaces. The tree-search version is **not complete**: on the Romania map it can loop forever (Arad → Sibiu → Arad → …). In infinite state spaces both versions fail if an infinite non-goal path is encountered.
- **Optimal:** No. If the left subtree contained a deep goal and the right subtree a shallow one, DFS would return the deep one.
- **Time:** O(b^m), where m is the maximum depth — terrible if m is much larger than d, and infinite if m is infinite.
- **Space:** O(bm) — DFS needs to store only a single path from root to leaf, plus the unexpanded siblings of each node on the path. With b = 10 and d = 16, where BFS needs 10 exabytes, DFS needs only about 156 kilobytes. A variant called **backtracking search** generates only one successor at a time and needs only O(m) memory.

Advantages of DFS: very low memory; may find a solution quickly if solutions are plentiful and deep; easy recursive implementation. **Disadvantages:** may get lost in infinite or very deep branches; not optimal; not complete for tree search.

Tip for answers: When asked to "trace BFS and DFS" on a given tree, always state your conventions first: children are generated left to right, the goal test is applied on expansion, and repeated states are/are not checked. Then give the order of expansion as a list and draw the frontier (queue or stack) at each step. Clear conventions prevent loss of marks when your order differs from the examiner's key.

Depth-Limited Search (DLS)

The failure of DFS in infinite state spaces can be fixed by supplying a predetermined **depth limit l**: nodes at depth l are treated as if they had no successors. This is **depth-limited search**.

The depth limit solves the infinite-path problem, but introduces another source of incompleteness: if l < d (the shallowest goal is beyond the limit), no solution is found. If l > d, DLS is not optimal. Depth limits can sometimes be based on knowledge of the problem. On the full 20-city Romania map, any city can be reached from any other in at most 9 steps — this number, the **diameter** of the state space, is a good depth limit.

DLS can terminate with two kinds of failure: **failure** means no solution exists at all; **cutoff** means no solution exists within the depth limit.

Algorithm

```
function DEPTH-LIMITED-SEARCH(problem, limit) returns solution, failure or cutoff
  return RECURSIVE-DLS(MAKE-NODE(problem.INITIAL-STATE), problem, limit)

function RECURSIVE-DLS(node, problem, limit) returns solution, failure or cutoff
  if problem.GOAL-TEST(node.STATE) then return SOLUTION(node)
  else if limit = 0 then return cutoff
  else
    cutoff_occurred? <- false
    for each action in problem.ACTIONS(node.STATE) do
      child  <- CHILD-NODE(problem, node, action)
      result <- RECURSIVE-DLS(child, problem, limit - 1)
      if result = cutoff then cutoff_occurred? <- true
      else if result ≠ failure then return result
    if cutoff_occurred? then return cutoff else return failure
```

Trace on the Example Tree

- With **l = 2**: DLS visits A, B, D, E, C, F, G. The nodes at depth 2 (D, E, F, G) are not expanded. K (depth 3) is never reached, so the result is **cutoff**.
- With **l = 3**: DLS visits A, B, D, H, I, E, J, C, F, K and succeeds — identical to DFS on this tree, because the tree has no node deeper than 3.

Properties of DLS

- **Complete:** Only if l ≥ d.
- **Optimal:** No (even when l ≥ d, it may return a deeper goal first).
- **Time:** O(b^l). **Space:** O(bl).
- DFS can be viewed as a special case of DLS with l = ∞.

Iterative Deepening Depth-First Search (IDS)

Iterative deepening search (iterative deepening depth-first search, IDDFS) is a general strategy, often used in combination with depth-first tree search, that finds the best depth limit. It does this by gradually increasing the limit — first 0, then 1, then 2, and so on — until a goal is found. This will happen when the depth limit reaches d, the depth of the shallowest goal node.

IDS combines the benefits of DFS and BFS: like DFS its memory requirement is modest, O(bd); like BFS it is complete when the branching factor is finite and optimal when the path cost is a non-decreasing function of depth.

Algorithm

```
function ITERATIVE-DEEPENING-SEARCH(problem) returns a solution or failure
  for depth = 0 to ∞ do
    result <- DEPTH-LIMITED-SEARCH(problem, depth)
    if result ≠ cutoff then return result
```

Trace on the Example Tree

Table 1.32: IDS iterations on the example tree

Iteration (limit)	Nodes visited in order	Count	Result
0	A	1	cutoff
1	A, B, C	3	cutoff
2	A, B, D, E, C, F, G	7	cutoff
3	A, B, D, H, I, E, J, C, F, K	10	goal K found
Total	—	21	Path A → C → F → K

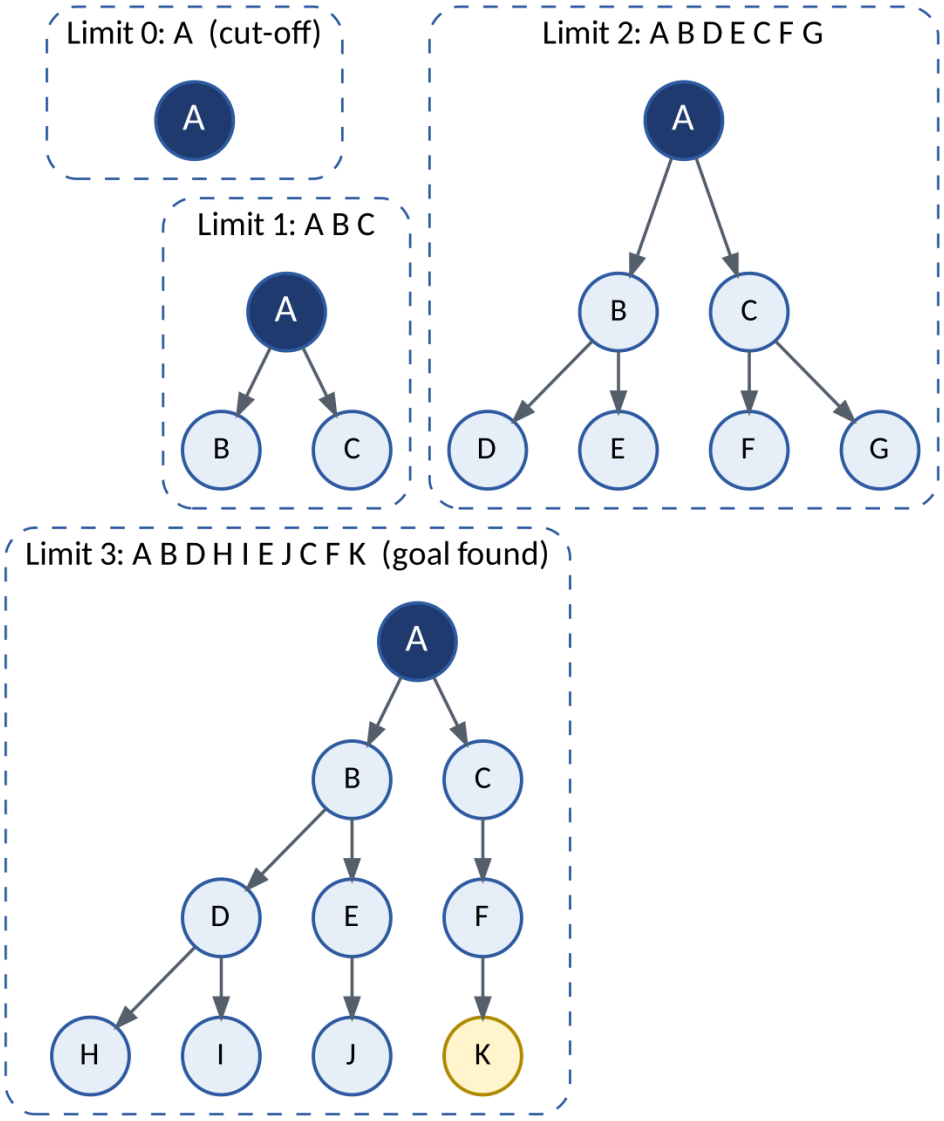


Figure 1.16: The four iterations of IDS on the example tree

Is IDS Wasteful?

IDS may seem wasteful because states near the root are generated many times. But in a search tree with the same (or nearly the same) branching factor at each level, most of the nodes are in the bottom level, so it does not matter much that the upper levels are repeated. In IDS, the nodes on the bottom level (depth d) are generated once, those on the next-to-bottom level twice, and so on, up to the children of the root, which are generated d times:

N(IDS) = (d)b + (d − 1)b^2 + … + (1)b^d, which is O(b^d).

For b = 10 and d = 5:

- N(IDS) = 50 + 400 + 3,000 + 20,000 + 100,000 = 123,450
- N(BFS) = 10 + 100 + 1,000 + 10,000 + 100,000 = 111,110

So IDS generates only about 11% more nodes than BFS, while using linear rather than exponential memory.

Properties of IDS

- **Complete:** Yes, if b is finite.
- **Optimal:** Yes, if all step costs are equal.
- **Time:** O(b^d). **Space:** O(bd).

> **Key Point:** In general, iterative deepening is the preferred uninformed search method when the search space is large and the depth of the solution is not known. It gets BFS's completeness and shallowest-solution guarantee for the memory price of DFS.

Bidirectional Search

The idea behind **bidirectional search** is to run two simultaneous searches — one **forward** from the initial state and the other **backward** from the goal — hoping that the two searches meet in the middle. The motivation is that b^(d/2) + b^(d/2) is much less than b^d. For example, if the solution depth is d = 6 and each direction runs breadth-first search, in the worst case the two searches meet when each has generated all nodes at depth 3. For b = 10 this means a total of 2,220 node generations, compared with 1,111,110 for a standard breadth-first search.

Bidirectional search is implemented by replacing the goal test with a check to see whether the frontiers of the two searches **intersect**; if they do, a solution has been found. The check can be done when each node is generated or selected for expansion, and with a hash table it takes constant time.

Trace: Arad to Bucharest (Both Directions Breadth-First, by Levels)

- **Forward level 1** (from Arad): Zerind, Sibiu, Timisoara. No intersection with {Bucharest}.
- **Backward level 1** (from Bucharest): Fagaras, Pitesti, Craiova. No intersection with the forward set.
- **Forward level 2**: Oradea, Fagaras, Rimnicu Vilcea, Lugoj. **Fagaras** is already in the backward set — the frontiers meet.
- **Solution:** join the forward path Arad → Sibiu → Fagaras with the reversed backward path Fagaras → Bucharest, giving Arad → Sibiu → Fagaras → Bucharest (3 steps, 450 km).

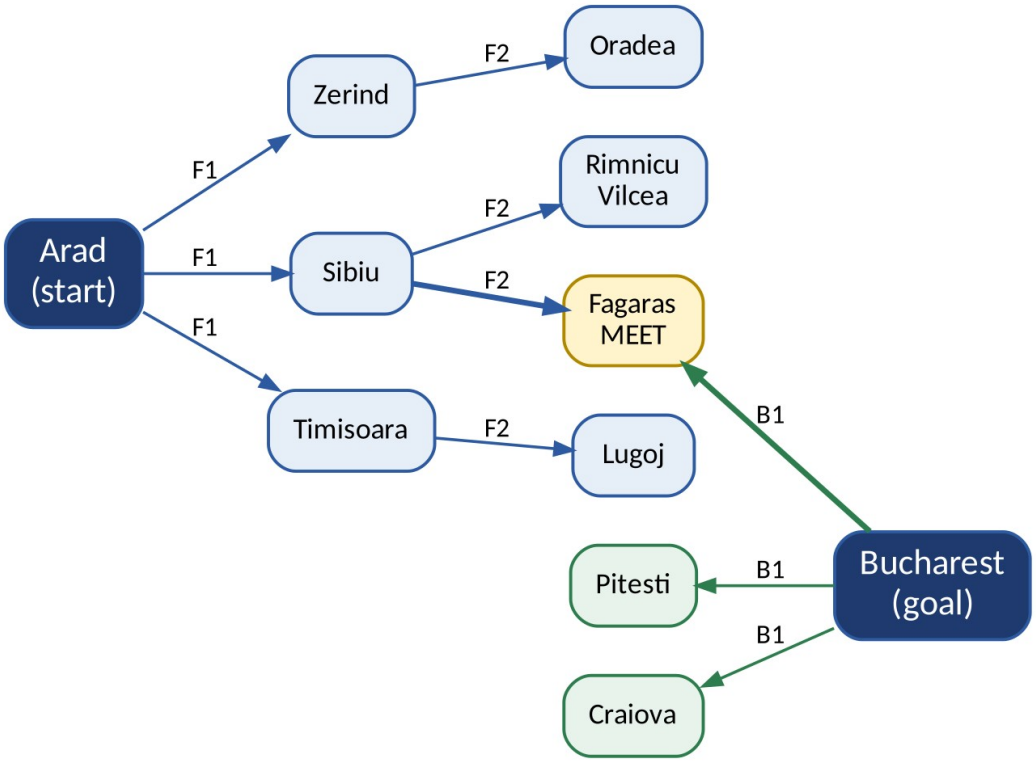


Figure 1.17: Bidirectional search from Arad and Bucharest meeting at Fagaras

The meeting point gives the path with the **fewest steps**, which is what BFS guarantees. To guarantee the least-cost route (418 km), both halves must be uniform-cost searches, and the stopping condition must be handled more carefully.

Properties of Bidirectional Search

- **Complete:** Yes, if b is finite and both directions use BFS.
- **Optimal:** Yes, if step costs are all equal and both directions use BFS.
- **Time:** O(b^(d/2)). **Space:** O(b^(d/2)) — at least one of the frontiers must be kept in memory for the intersection check, which is the main weakness.

Difficulties

- Searching backward requires computing **predecessors**: states that have a given state as successor. This is easy when actions are reversible (route finding, 8-puzzle) but can be very hard otherwise.
- There must be an **explicit goal state** (or a small set of them). If the goal is described by a property, such as "checkmate" or "no queens attack each other", it is unclear where to start the backward search.
- The intersection test must be efficient.

Comparing Uninformed Search Strategies

Table 1.33: Comparison of uninformed search strategies (tree-search versions)

Criterion	BFS	Uniform-cost	DFS	Depth-limited	Iterative deepening	Bidirectional (BFS)
Complete?	Yes (a)	Yes (a, b)	No	No	Yes (a)	Yes (a, d)

Criterion	BFS	Uniform-cost	DFS	Depth-limited	Iterative deepening	Bidirectional (BFS)
Optimal?	Yes (c)	Yes	No	No	Yes (c)	Yes (c, d)
Time	O(b^d)	O(b^(1+⌊C*/ε⌋))	O(b^m)	O(b^l)	O(b^d)	O(b^(d/2))
Space	O(b^d)	O(b^(1+⌊C*/ε⌋))	O(bm)	O(bl)	O(bd)	O(b^(d/2))
Frontier data structure	FIFO queue	Priority queue on g(n)	LIFO stack	LIFO stack + limit	Repeated DLS	Two queues

Notes on the table: (a) complete if b is finite; (b) complete if step costs are at least ε > 0; (c) optimal if step costs are all identical; (d) if both directions use breadth-first search. b = branching factor, d = depth of shallowest solution, m = maximum depth of the search tree, l = depth limit, C^* = cost of the optimal solution, ε = smallest step cost.

BFS versus DFS

Table 1.34: BFS compared with DFS

Point	Breadth-first search	Depth-first search
Data structure	FIFO queue	LIFO stack (or recursion)
Order	Level by level	Deepest node first, then backtrack
Completeness	Complete (finite b)	Incomplete in infinite or looping spaces
Optimality	Optimal for equal step costs	Not optimal
Time	O(b^d)	O(b^m)
Space	O(b^d) — main drawback	O(bm) — main advantage
Best when	Solution is shallow; memory is plentiful	Many deep solutions; memory is limited
Risk	Running out of memory	Getting lost down an infinite branch

Memory Trick: "Queue for Breadth, Stack for Depth, Priority for Price" — BFS uses a queue, DFS a stack, UCS a priority queue on path cost. And IDS is "DFS wearing BFS's guarantees".

Worked Example — Choosing a strategy: (1) A puzzle with b = 3, solutions known to be at most 20 moves deep, and limited memory: use IDS — complete, optimal for unit costs, linear memory. (2) Delivery routing on a road network with distances: use uniform-cost search, because step costs differ. (3) Finding any path out of a maze with no loops and a few very deep exits: DFS is fine and uses very little memory. (4) Finding the shortest chain of friends between two people in a social network where links work both ways: bidirectional BFS, because both ends are known and actions are reversible.

Avoiding Repeated States

In state spaces with reversible actions (route finding, sliding puzzles, water jug), a tree search can generate the same state many times, and may even loop forever. Three levels of protection are used, in increasing cost:

1. Do not return to the state you just came from (do not generate the parent as a child).
2. Do not create paths with cycles (do not generate any ancestor of the node).

3. Do not generate any state that has ever been generated before — keep an **explored set** (closed list), which turns tree search into **graph search**.

The third option is the most effective but requires memory proportional to the number of states visited. For the water-jug problem, graph search explores only the 14 reachable states, while tree search would generate an infinite tree because of moves such as fill-then-empty.

Introduction to Informed (Heuristic) Search

Uninformed strategies are systematic but blind: they spend as much effort on hopeless directions as on promising ones. **Informed search** uses a **heuristic function h(n)** — an estimate of the cost of the cheapest path from node n to a goal — to focus the search. h(n) depends only on the state at n, and h(goal) = 0.

For route finding in Romania a good heuristic is the **straight-line distance** to Bucharest, hSLD. Some values: Arad 366, Zerind 374, Timisoara 329, Sibiu 253, Oradea 380, Fagaras 176, Rimnicu Vilcea 193, Pitesti 100, Craiova 160, Bucharest 0.

- **Greedy best-first search** expands the node that appears closest to the goal, using f(n) = h(n). From Arad it chooses Sibiu (253, lower than Timisoara 329 and Zerind 374), then Fagaras (176, lower than Rimnicu Vilcea 193), then Bucharest (0). It finds Arad → Sibiu → Fagaras → Bucharest at 450 km after expanding only three nodes — fast, but **not optimal**.
- **A* search** uses f(n) = g(n) + h(n), the estimated cost of the cheapest solution through n. It expands Arad (f = 366), Sibiu (393), Rimnicu Vilcea (413), Fagaras (415), Pitesti (417) and then Bucharest (418), returning the optimal 418 km route. A* is optimal when h never overestimates the true cost (an **admissible** heuristic).

Table 1.35: Uninformed versus informed search

Feature	Uninformed (blind) search	Informed (heuristic) search
Knowledge used	Only the problem definition	Problem definition plus heuristic h(n)
Evaluation of nodes	Order of generation or g(n)	h(n) or g(n) + h(n)
Efficiency	Often exponential; explores blindly	Usually far fewer nodes
Examples	BFS, UCS, DFS, DLS, IDS, bidirectional	Greedy best-first, A*, hill climbing

These methods, the design of heuristics and their properties are developed fully in Unit 2.

Generate-and-Test

Generate-and-test is the simplest of all search strategies.

```
procedure GENERATE-AND-TEST
  1. Generate a possible solution
     (a point in the problem space, or a path from the start state).
  2. Test whether it is actually a solution: compare it with the goal
     or check the goal conditions.
  3. If a solution has been found, quit. Otherwise return to step 1.
```

If the generation is **systematic** (every candidate is eventually produced, none twice), the procedure is guaranteed to find a solution if one exists — it is then essentially an exhaustive depth-first search of the candidate space, sometimes called the **British Museum algorithm** when it is fully exhaustive. If candidates are generated **randomly**, there is no guarantee.

- The four-city TSP worked example in the previous chapter is generate-and-test: generate each of the three tours, compute its length, keep the shortest.

- For 8-queens, generate each of the 8! = 40,320 permutations (one queen per row and column) and test the diagonals; exactly 92 pass.
- DENDRAL used a refined version, **plan-generate-test**: a planning step uses constraint knowledge to limit which structures are generated, so the generator produces far fewer candidates.

Generate-and-test is acceptable for small problems but hopeless for large ones, because the number of candidates grows exponentially. Its usefulness rises sharply when heuristics guide the generator — which leads to hill climbing.

Hill Climbing

Hill climbing is generate-and-test with feedback: the test function supplies a heuristic value telling the generator which direction to move. It is a **local search** method: it keeps only the current state, looks at its neighbours, and moves to a better one. It does not maintain a search tree, so memory use is constant.

```
function HILL-CLIMBING(problem) returns a state that is a local maximum
  current <- MAKE-NODE(problem.INITIAL-STATE)
  loop do
    neighbour <- a highest-valued successor of current
    if neighbour.VALUE ≤ current.VALUE then return current.STATE
    current <- neighbour
```

This version is **steepest-ascent** hill climbing: it always moves to the best neighbour. **Simple** hill climbing moves to the first neighbour that is better than the current state. When minimising a cost (such as the number of attacking queen pairs in the complete-state 8-queens formulation), the method is sometimes called **gradient descent**.

Example: hill climbing on a one-dimensional landscape. Let x range over 0 to 8 with values f(0..8) = 1, 3, 5, 4, 2, 6, 9, 7, 3, and neighbours x − 1 and x + 1. Starting at x = 0: move to x = 1 (3), then x = 2 (5); both neighbours of x = 2 have lower values (3 and 4), so the algorithm stops with f = 5 — a local maximum, although the global maximum is f(6) = 9. A random restart at x = 4 (value 2) moves to x = 5 (6) and then x = 6 (9), where both neighbours (6 and 7) are lower, so it stops at the global maximum.

Problems with Hill Climbing

Table 1.36: Problems with hill climbing and their remedies

Problem	Description	Remedy
Local maximum	A peak higher than its neighbours but lower than the global maximum	Backtrack, or random-restart hill climbing
Plateau	A flat area where all neighbours have the same value; no direction to move	Allow sideways moves (with a limit) or make a big jump
Ridge	A sequence of local maxima that is hard to climb because every single move goes downhill	Apply several moves at once; move in several directions

Russell and Norvig report that, starting from a randomly generated 8-queens state, steepest-ascent hill climbing gets stuck 86% of the time and solves only 14% of problem instances — but it works quickly, taking only about four steps on average when it succeeds. **Random-restart hill climbing** ("if at first you don't succeed, try, try again") conducts a series of hill-climbing searches from random initial states and is complete with probability approaching 1. **Stochastic hill climbing** chooses at random among uphill moves; **simulated annealing**, covered later, occasionally allows downhill moves to escape local maxima.

Exam Tip: Draw a simple "state-space landscape" curve with labelled global maximum, local maximum, plateau (flat shoulder) and current state when explaining hill climbing. A sketch with these four labels plus the remedies table is enough for a 5-mark answer.

Summary

- Search strategies are evaluated by completeness, optimality, time and space, using b (branching factor), d (depth of shallowest goal) and m (maximum depth).
- BFS (FIFO queue) is complete and optimal for equal step costs but needs O(b^d) memory.
- UCS (priority queue on g) is complete and optimal for any positive step costs; it applies the goal test on expansion.
- DFS (stack) needs only O(bm) memory but is neither complete (in general) nor optimal; DLS adds a depth limit l.
- IDS repeats DLS with limits 0, 1, 2, ... and is complete, optimal for unit costs, O(b^d) time and O(bd) space — the preferred uninformed method.
- Bidirectional search meets in the middle in O(b^(d/2)) but needs predecessors and an explicit goal.
- Informed search uses a heuristic h(n): greedy best-first uses f(n) = h(n); A* uses f(n) = g(n) + h(n).
- Generate-and-test is exhaustive trial; hill climbing adds heuristic feedback but suffers from local maxima, plateaux and ridges.

Unit 1 Cheat Sheet

Core Definitions

Table 1.37: Unit 1 key definitions

Term	One-line definition
Intelligence	Ability to learn, reason, solve problems, perceive, use language and adapt to achieve goals
Artificial intelligence	Science and engineering of making intelligent machines (McCarthy); named at Dartmouth, 1956
Turing test (1950)	Machine passes if an interrogator cannot tell it from a human by typed conversation
Total Turing test	Turing test plus video (vision) and object passing (robotics)
Chinese room (Searle, 1980)	Symbol manipulation (syntax) is not understanding (semantics)
Weak AI / Strong AI	Machine acts as if intelligent / machine really has a mind
Narrow AI / General AI	Good at one task / human-level across most tasks
Agent	Perceives through sensors, acts through actuators
Rational agent	Chooses action maximising expected performance measure given percepts and knowledge
PEAS	Performance measure, Environment, Actuators, Sensors
State space	All states reachable from the initial state; a graph of states and actions
Node	Search-tree data structure: state, parent, action, path cost g(n), depth
Frontier	Generated but not yet expanded nodes (open list)
Explored set	Already expanded states (closed list)

Four Approaches to AI

Table 1.38: Four approaches in one grid

	Human-like	Rational
Thinking	Thinking humanly — cognitive modelling, GPS	Thinking rationally — laws of thought, logic
Acting	Acting humanly — Turing test	Acting rationally — rational agents (modern standard)

- Turing test needs: NLP, knowledge representation, automated reasoning, machine learning (+ vision, robotics for total test).

Foundations in One Line Each

- **Philosophy:** logic (Aristotle), mind–body (Descartes), induction (Hume), knowledge → action.
- **Mathematics:** Boolean and first-order logic, computability (Turing, Gödel), NP-completeness, probability (Bayes).

- **Economics:** utility, decision theory, game theory (von Neumann and Morgenstern 1944), satisficing (Simon).
- **Neuroscience:** neurons (Golgi, Cajal), localisation (Broca 1861), inspiration for neural nets.
- **Psychology:** behaviourism, cognitive psychology (Craik 1943), cognitive science.
- **Computer engineering:** programmable computers, Lisp (1958), GPUs.
- **Control theory:** feedback, cybernetics (Wiener 1948), objective functions.
- **Linguistics:** Chomsky's formal grammars (1957), NLP.

History Timeline

Table 1.39: History of AI at a glance

Year	Event
1943	McCulloch–Pitts artificial neuron
1950	Turing's paper; Turing test
1956	Dartmouth workshop — AI named; Logic Theorist
1957–58	GPS, Lisp, perceptron
1966–73	ALPAC, *Perceptrons* (1969), Lighthill (1973) — first AI winter
1970s	DENDRAL, MYCIN — expert systems
1982	R1/XCON commercial success
1986	Back-propagation popularised
Late 1980s	Second AI winter
1997	Deep Blue beats Kasparov
2012	AlexNet — deep-learning era
2016–17	AlphaGo; Transformer
2020–22	GPT-3; ChatGPT — LLM era

Problem Formulation

- Five components: **initial state, actions (operators), transition model RESULT(s, a), goal test, path cost** (sum of step costs c(s, a, s′)).
- Initial state + actions + transition model = state space.
- Problem-solving agent: formulate goal → formulate problem → search → execute (open loop).
- Problem types: **single-state** (sequence), **sensorless/conformant** (belief states), **contingency** (conditional plan), **exploration** (unknown space).
- Production system = rules + database + control strategy + rule applier; control must cause motion and be systematic.

Classic Problems

Table 1.40: Classic problems summary

Problem	State	Actions	Key fact
Vacuum world	(location, dirt A, dirt B)	Left, Right, Suck	8 states; sensorless plan [R, S, L, S]

Problem	State	Actions	Key fact
8-puzzle	Positions of 8 tiles + blank	Blank Up/Down/Left/Right	9!/2 = 181,440 reachable states
8-queens	Queens on board	Add / move queen	2,057 states (improved); 92 solutions
Water jug (4, 3 → 2)	(x, y)	12 production rules	Shortest solution 6 steps
Missionaries and cannibals	(M, C, B) on left bank	Move 1–2 people	11 crossings
Romania route	City	Go(neighbour)	BFS 450 km (3 steps); UCS 418 km
TSP	(city, visited set)	Go to unvisited city	(n − 1)!/2 tours

- Water-jug path: (0,0) → (0,3) → (3,0) → (3,3) → (4,2) → (0,2) → (2,0).
- Alternative: (0,0) → (4,0) → (1,3) → (1,0) → (0,1) → (4,1) → (2,3).

Search Strategies in One Line

- **BFS:** FIFO queue; shallowest first; complete; optimal for equal costs; O(b^d) time and space.
- **UCS:** priority queue on g(n); goal test on expansion; optimal; O(b^(1+⌊C*/ε⌋)).
- **DFS:** stack; deepest first; not complete, not optimal; O(b^m) time, O(bm) space.
- **DLS:** DFS with limit l; complete only if l ≥ d; returns failure or cutoff.
- **IDS:** DLS with l = 0, 1, 2, ...; complete; optimal for unit costs; O(b^d) time, O(bd) space.
- **Bidirectional:** forward + backward, meet in middle; O(b^(d/2)); needs predecessors and explicit goal.
- **Greedy best-first:** f(n) = h(n). **A*:** f(n) = g(n) + h(n).
- **Generate-and-test:** generate candidate, test, repeat. **Hill climbing:** move to best neighbour; stuck at local maxima, plateaux, ridges.

Table 1.41: Search comparison (for equal step costs)*

	BFS	UCS	DFS	DLS	IDS	Bidir.
Complete	Yes	Yes	No	No	Yes	Yes
Optimal	Yes*	Yes	No	No	Yes*	Yes*
Time	b^d	b^(1+C*/ε)	b^m	b^l	b^d	b^(d/2)
Space	b^d	b^(1+C*/ε)	bm	bl	bd	b^(d/2)

- Example tree orders: BFS A B C D E F G H I J K; DFS A B D H I E J C F K; IDS total 1 + 3 + 7 + 10 = 21 visits.
- IDS overhead: b = 10, d = 5 → 123,450 vs BFS 111,110 nodes (about 11% more).

Memory Tricks

- **HT-RT-HA-RA:** Human Thinking, Rational Thinking, Human Acting, Rational Acting.
- **"People Make Excellent New Plans; Computers Can Learn":** Philosophy, Mathematics, Economics, Neuroscience, Psychology, Computer engineering, Control theory, Linguistics.
- **PEAS:** Performance, Environment, Actuators, Sensors.
- **"I Am The Great Planner":** Initial state, Actions, Transition model, Goal test, Path cost.
- **COTS:** Completeness, Optimality, Time, Space.
- **"Queue for Breadth, Stack for Depth, Priority for Price."**

- **"S-S-C-E":** Single-state, Sensorless, Contingency, Exploration — from most known to least known.
- **"L-P-R" hill-climbing traps:** Local maximum, Plateau, Ridge.
- **Water jug 4-3-2:** "Fill 3, pour, fill 3, top up, empty 4, pour" — six steps.

Important Questions

Two-Mark Questions (with Model Answers)

1. Define artificial intelligence. AI is the science and engineering of making intelligent machines, especially intelligent computer programs (McCarthy). Rich and Knight: "the study of how to make computers do things at which, at the moment, people are better."

2. What is the Turing test? A test proposed by Alan Turing in 1950: a human interrogator holds typed conversations with a hidden human and a hidden machine. If the interrogator cannot reliably tell which is the machine, the machine is said to pass.

3. What is the total Turing test? An extension of the Turing test that includes a video signal and the passing of physical objects, so the machine additionally needs computer vision and robotics.

4. State the Chinese room argument in one or two lines. Searle (1980) imagined a non-Chinese speaker answering Chinese questions by following a rulebook. The room appears to understand Chinese, but no understanding exists — so symbol manipulation (syntax) is not sufficient for meaning (semantics).

5. Distinguish strong AI and weak AI. Weak AI claims machines can act as if they are intelligent. Strong AI claims a suitably programmed machine actually has a mind and understands.

6. List the four approaches to AI. Thinking humanly (cognitive modelling), thinking rationally (laws of thought), acting humanly (Turing test) and acting rationally (rational agents).

7. Name any four disciplines that form the foundations of AI. Philosophy, mathematics, economics, neuroscience (also psychology, computer engineering, control theory and linguistics).

8. Why is 1956 important in AI history? The Dartmouth summer workshop organised by McCarthy, Minsky, Shannon and Rochester was held in 1956; the term "artificial intelligence" was adopted and the field was formally born.

9. What is an AI winter? A period of reduced funding and interest in AI after expectations were not met. The first was around 1966–1973 (ALPAC, Lighthill report); the second in the late 1980s (expert-system and Lisp-machine collapse).

10. What is PEAS? A framework for specifying a task environment: Performance measure, Environment, Actuators, Sensors. For a vacuum agent: cleanliness; squares A and B; Left/Right/Suck; location and dirt sensors.

11. List the components of a well-defined problem. Initial state, actions (operators), transition model, goal test and path cost.

12. Define state space. The set of all states reachable from the initial state by any sequence of actions, forming a graph whose nodes are states and whose arcs are actions.

13. What is a sensorless (conformant) problem? A problem in which the agent has no sensors and does not know its exact state; it searches belief states for an action sequence that reaches the goal from every possible state.

14. Define branching factor and give the space complexity of DFS. The branching factor b is the maximum number of successors of any node. DFS needs O(bm) space, where m is the maximum depth.

15. Why is iterative deepening search preferred among uninformed methods? It is complete and optimal for unit step costs like BFS, but needs only O(bd) memory like DFS; the repeated work at shallow levels is small (about 11% extra for b = 10).

Five-Mark Questions (with Answer Outlines)

1. Explain the four approaches to AI with a diagram. Outline: draw the 2 × 2 grid (thought/behaviour × human/rational); thinking humanly — cognitive modelling, GPS, cognitive science; thinking rationally — Aristotle's syllogisms, logicist tradition and its two obstacles; acting humanly — Turing test and its four capabilities; acting rationally — rational agent, why it is preferred (more general, mathematically well defined).

2. Describe the Turing test and its limitations. Outline: 1950 paper, imitation game; diagram with interrogator, human, machine; four required capabilities; total Turing test adds vision and robotics; limitations — tests human-likeness not intelligence, can be fooled by tricks (ELIZA), says nothing about understanding (Chinese room), AI researchers focus on principles instead ("artificial flight" analogy).

3. Explain the problem-solving agent with its algorithm. Outline: goal-based agent, atomic states; four phases — goal formulation, problem formulation, search, execution; Romania holiday example; flow diagram; SIMPLE-PROBLEM-SOLVING-AGENT pseudocode; open-loop execution and when it is safe.

4. Explain the different types of problems with examples. Outline: single-state (fully observable, deterministic; sequence solution); sensorless/conformant (belief states; vacuum [Right, Suck, Left, Suck]); contingency (partially observable or non-deterministic; conditional plan; erratic vacuum); exploration (unknown state space; online search). Add toy vs real-world problems with examples.

5. Formulate the 8-puzzle problem and draw part of its state space. Outline: states (positions of tiles and blank), initial state, actions (blank Up/Down/Left/Right), transition model, goal test, path cost 1 per move; 181,440 reachable states; draw a two-level tree from a sample state; mention repeated states.

6. Explain breadth-first search with an example. State its properties. Outline: FIFO queue; pseudocode; trace table on a small tree showing the queue at each step; order of expansion; properties — complete, optimal for equal costs, O(b^d) time and space; memory table showing the problem; advantages and disadvantages.

7. Explain depth-limited and iterative deepening search. Outline: DLS — depth limit l, cutoff vs failure, complete only if l ≥ d, O(b^l), O(bl); IDS — loop over limits, trace with iterations, node-count formula N(IDS) = (d)b + (d − 1)b^2 + … + b^d, b = 10, d = 5 example (123,450 vs 111,110); properties.

8. What is hill climbing? Explain its problems. Outline: local search, generate-and-test with heuristic feedback; algorithm; steepest-ascent vs simple; landscape sketch; local maximum, plateau, ridge with remedies; random restart; mention 8-queens behaviour.

Ten-Mark Questions (with Structured Answer Plans)

1. Explain the foundations and history of artificial intelligence. Plan: (a) Introduction — AI defined, McCarthy, 1956. (b) Foundations diagram and eight short paragraphs: philosophy, mathematics, economics, neuroscience, psychology, computer engineering, control theory, linguistics — each with its key question, contribution and one name. (c) History by phases with dates: 1943 McCulloch–Pitts; 1950 Turing; 1956 Dartmouth; early enthusiasm (GPS, Lisp, perceptron); first AI winter (ALPAC 1966, *Perceptrons* 1969, Lighthill 1973); expert systems (DENDRAL, MYCIN, R1); second AI winter; back-propagation 1986; Deep Blue 1997; AlexNet 2012; AlphaGo 2016; Transformer 2017; LLMs and ChatGPT 2022. (d) Timeline diagram. (e) Conclusion — each comeback came from a new idea plus more computing power and data.

2. (Numerical) You are given a 5-litre jug and a 3-litre jug, with no markings, and a tap. How can you get exactly 4 litres in the 5-litre jug? Formulate the problem, write the production rules and give the solution path.

Solution. State (x, y): x = litres in the 5-litre jug (0–5), y = litres in the 3-litre jug (0–3). Initial state (0, 0). Goal state (4, n) for any n. Path cost: 1 per operation.

Table 1.42: Production rules for the 5-litre and 3-litre jug problem

Rule	Condition	New state	Meaning
R1	x < 5	(5, y)	Fill the 5-litre jug
R2	y < 3	(x, 3)	Fill the 3-litre jug
R3	x > 0	(0, y)	Empty the 5-litre jug
R4	y > 0	(x, 0)	Empty the 3-litre jug
R5	x + y ≥ 5, y > 0	(5, y − (5 − x))	Pour 3 → 5 until 5-litre jug is full
R6	x + y ≥ 3, x > 0	(x − (3 − y), 3)	Pour 5 → 3 until 3-litre jug is full
R7	x + y ≤ 5, y > 0	(x + y, 0)	Pour all of 3-litre into 5-litre
R8	x + y ≤ 3, x > 0	(0, x + y)	Pour all of 5-litre into 3-litre

Table 1.43: Solution path for the 5-litre and 3-litre jug problem

Step	x (5 L)	y (3 L)	Rule	Action
0	0	0	—	Start
1	5	0	R1	Fill 5-litre jug
2	2	3	R6	Pour 5 → 3 until 3-litre is full
3	2	0	R4	Empty 3-litre jug
4	0	2	R8	Pour all of 5-litre into 3-litre
5	5	2	R1	Fill 5-litre jug
6	4	3	R6	Pour 5 → 3 until full (1 litre moves) — goal

The goal (4, 3) is reached in six operations; a breadth-first search confirms no shorter solution exists. Complete the answer with a state-space sketch of the six steps and a note that BFS as the control strategy guarantees the shortest path.

3. (Numerical) For the graph below, apply BFS, DFS, IDS and uniform-cost search from S to goal G. Give the order of expansion, the path found and its cost. Edges (directed, with costs): S → A (1), S → B (5), S → C (15), A → G (10), B → G (5), C → G (5).

Solution (children generated in alphabetical order; goal test on expansion; tree-search versions, so a state may appear more than once on the frontier).

```
BFS:  queue [S] -> expand S -> [A, B, C]
      expand A -> [B, C, G(via A)]
      expand B -> [C, G(via A), G(via B)]
      expand C -> [G(via A), G(via B), G(via C)]
      expand G(via A) -> goal
      Order: S, A, B, C, G     Path: S-A-G     Cost: 11

DFS:  stack [S] -> pop S, push C, B, A -> pop A, push G -> pop G -> goal
      Order: S, A, G           Path: S-A-G     Cost: 11
```

```
IDS:  limit 0: S
      limit 1: S, A, B, C
      limit 2: S, A, G -> goal
      Path: S-A-G     Cost: 11

UCS:  frontier {S:0}
      expand S(0)  -> {A:1, B:5, C:15}
      expand A(1)  -> {B:5, G:11, C:15}
      expand B(5)  -> G via B costs 10 < 11, replace -> {G:10, C:15}
      expand G(10) -> goal
      Order: S, A, B, G        Path: S-B-G     Cost: 10
```

Conclusion: BFS, DFS and IDS return S-A-G (cost 11), because they ignore step costs; only UCS finds the optimal path S-B-G (cost 10). Complete the answer with the properties table for the four strategies.

4. Explain uninformed search strategies and compare them. Plan: (a) Define search strategy and uninformed search; node data structure; frontier types. (b) Performance measures: completeness, optimality, time, space; b, d, m. (c) For each of BFS, UCS, DFS, DLS, IDS and bidirectional: idea, data structure, short algorithm, one trace (use the example tree A–L with goal K and the Romania map), properties. (d) Diagrams: example tree with BFS and DFS orders, IDS iterations, bidirectional meeting. (e) Comparison table with footnotes. (f) Conclusion: IDS preferred for large spaces of unknown depth; UCS when costs differ; informed search (A*) as the next step.

MCQ Practice

Choose the single best answer for each question. The answer key with explanations follows the questions.

1. The term "artificial intelligence" was first used in connection with: (a) Turing's 1950 paper (b) The 1956 Dartmouth workshop proposal (c) The Lighthill report (d) The Logic Theorist manual

2. Who proposed the imitation game now called the Turing test? (a) John McCarthy (b) Marvin Minsky (c) Alan Turing (d) Herbert Simon

3. Which capability is NOT required for the standard (non-total) Turing test? (a) Natural language processing (b) Knowledge representation (c) Robotics (d) Machine learning

4. The Chinese room argument was proposed by: (a) John Searle (b) Noam Chomsky (c) Hubert Dreyfus (d) Alan Turing

5. The approach to AI based on the Turing test is: (a) Thinking humanly (b) Thinking rationally (c) Acting humanly (d) Acting rationally

6. Aristotle's syllogisms belong to which approach? (a) Acting humanly (b) Thinking rationally (c) Acting rationally (d) Thinking humanly

7. The claim that a suitably programmed machine actually has a mind is called: (a) Weak AI (b) Narrow AI (c) Strong AI (d) Applied AI

8. Which discipline contributed decision theory and game theory to AI? (a) Linguistics (b) Economics (c) Neuroscience (d) Control theory

9. The first work generally recognised as AI (1943) was by: (a) Newell and Simon (b) McCulloch and Pitts (c) Rumelhart and Hinton (d) Minsky and Papert

10. The book *Perceptrons* (1969) contributed to: (a) The rise of expert systems (b) A decline in neural network research (c) The invention of Lisp (d) The Dartmouth workshop

11. MYCIN was an expert system for: (a) Configuring computers (b) Diagnosing blood infections (c) Inferring molecular structure (d) Playing chess

12. Deep Blue defeated Garry Kasparov in: (a) 1989 (b) 1997 (c) 2005 (d) 2011

13. The deep-learning era is usually dated from AlexNet's success in: (a) 2006 (b) 2010 (c) 2012 (d) 2016

14. In PEAS, the letter A stands for: (a) Agent (b) Actions (c) Actuators (d) Algorithm

15. An environment in which the next state is completely determined by the current state and action is: (a) Stochastic (b) Deterministic (c) Episodic (d) Dynamic

16. Which is NOT one of the five components of a problem formulation? (a) Initial state (b) Goal test (c) Heuristic function (d) Path cost

17. A problem in which the agent has no sensors but actions are deterministic is called: (a) Single-state (b) Sensorless (conformant) (c) Contingency (d) Exploration

18. How many states does the two-square vacuum world have? (a) 4 (b) 6 (c) 8 (d) 16

19. The number of reachable states in the 8-puzzle from a given state is: (a) 9! (b) 9!/2 (c) 8! (d) 2^9

20. In the 4-litre and 3-litre water-jug problem, the state (4, 2) is reached from (3, 3) by: (a) Filling the 4-litre jug (b) Pouring from the 3-litre into the 4-litre jug until full (c) Emptying the 3-litre jug (d) Pouring all of the 4-litre jug into the 3-litre jug

21. The minimum number of crossings in the 3-missionaries, 3-cannibals problem is: (a) 9 (b) 11 (c) 13 (d) 7

22. Breadth-first search uses which data structure for the frontier? (a) Stack (b) FIFO queue (c) Priority queue (d) Hash table only

23. The space complexity of depth-first search is: (a) O(b^d) (b) O(b^m) (c) O(bm) (d) O(b^(d/2))

24. Uniform-cost search expands the node with the lowest: (a) Depth (b) h(n) (c) g(n) (d) g(n) + h(n)

25. Depth-limited search is complete only when: (a) l < d (b) l ≥ d (c) l = 0 (d) b is infinite

26. Iterative deepening search has space complexity: (a) O(bd) (b) O(b^d) (c) O(b^m) (d) O(d)

27. Bidirectional search has time complexity: (a) O(b^d) (b) O(b^(2d)) (c) O(b^(d/2)) (d) O(bd)

28. On the example tree with goal K, DFS (left to right) expands nodes in the order: (a) A B C D E F G H I J K (b) A B D H I E J C F K (c) A C F K (d) A B D E C F G K

29. Greedy best-first search evaluates nodes using: (a) f(n) = g(n) (b) f(n) = h(n) (c) f(n) = g(n) + h(n) (d) f(n) = depth(n)

30. Which is NOT a problem with hill climbing? (a) Local maxima (b) Plateaux (c) Ridges (d) Exponential memory use

Answer Key

Table 1.44: MCQ answer key

Q	Answer	Explanation
1	b	McCarthy's 1955 proposal for the 1956 Dartmouth workshop introduced the term.
2	c	Turing described the imitation game in "Computing Machinery and Intelligence" (1950).
3	c	Robotics (and vision) are needed only for the total Turing test.
4	a	Searle published the Chinese room in 1980.
5	c	The Turing test judges human-like behaviour: acting humanly.
6	b	Syllogisms are "laws of thought" — the thinking rationally approach.
7	c	Strong AI claims real minds; weak AI claims only intelligent-seeming behaviour.
8	b	Economics gave utility, decision theory and game theory (von Neumann and Morgenstern).
9	b	McCulloch and Pitts proposed the artificial neuron model in 1943.
10	b	Minsky and Papert showed limits of single-layer perceptrons; funding for neural nets fell.
11	b	MYCIN diagnosed blood infections with about 450 rules; DENDRAL did molecular structure.

Q	Answer	Explanation
12	b	IBM's Deep Blue won the match in 1997.
13	c	AlexNet won the ImageNet competition in 2012.
14	c	PEAS = Performance, Environment, Actuators, Sensors.
15	b	Deterministic means outcomes are fully determined by state and action.
16	c	A heuristic is used by informed search, not part of the problem definition.
17	b	No sensors + deterministic actions = sensorless (conformant) problem.
18	c	2 locations × 2 × 2 dirt states = 8.
19	b	Only half of the 9! arrangements are reachable from any state: 181,440.
20	b	Pouring 1 litre from the 3-litre jug fills the 4-litre jug, leaving 2 litres (rule R7).
21	b	BFS confirms the optimal solution takes 11 crossings.
22	b	BFS expands the shallowest node, using a FIFO queue.
23	c	DFS stores one path plus unexpanded siblings: O(bm).
24	c	UCS orders the frontier by path cost g(n).
25	b	The goal must lie within the depth limit, so l ≥ d.
26	a	IDS runs DFS-style searches, needing only O(bd) memory.
27	c	Two searches each go to depth d/2.
28	b	DFS goes deep first and backtracks: A B D H I E J C F K.
29	b	Greedy best-first uses only the heuristic estimate h(n).
30	d	Hill climbing keeps only the current state, so memory is constant.

Glossary

Table 1.45: Glossary of Unit 1 terms

Term	Meaning
A* search	Informed search expanding the node with lowest f(n) = g(n) + h(n)
Abstraction	Removing irrelevant detail from a problem description
Actuator	Part of an agent through which it acts on the environment
Agent	Anything that perceives through sensors and acts through actuators
AI winter	Period of reduced funding and interest in AI after unmet expectations
Belief state	Set of physical states the agent might be in
Bidirectional search	Simultaneous forward and backward searches that meet in the middle
Branching factor (b)	Maximum number of successors of any node
Breadth-first search	Expands shallowest nodes first using a FIFO queue
Chinese room	Searle's argument that symbol manipulation is not understanding
Completeness	Guarantee of finding a solution whenever one exists
Contingency problem	Problem needing a conditional plan due to partial observability or non-determinism
Cutoff	DLS result meaning no solution within the depth limit
Depth-first search	Expands the deepest node first using a stack
Depth-limited search	DFS with a fixed depth limit l
Expert system	Program using expert knowledge rules for a narrow domain (e.g. MYCIN)
Exploration problem	Problem in which the state space is unknown and must be discovered
Explored set	Set of states already expanded (closed list)
Frontier	Set of generated nodes awaiting expansion (open list, fringe)
Generate-and-test	Repeatedly generate a candidate solution and test it
Goal test	Function deciding whether a state is a goal
Heuristic function h(n)	Estimated cost of cheapest path from n to a goal
Hill climbing	Local search that moves to the best neighbouring state
Iterative deepening search	Repeated DLS with limits 0, 1, 2, … until a goal is found
Local maximum	Peak higher than neighbours but lower than the global maximum
Node	Search-tree structure holding state, parent, action, path cost and depth

Term	Meaning
Operator	An action that transforms one state into another
Path cost	Numeric cost of a path, usually the sum of step costs
PEAS	Performance measure, Environment, Actuators, Sensors
Plateau	Flat region of the state-space landscape
Production rule	Condition → action rule in a production system
Rational agent	Agent that maximises expected performance given its percepts and knowledge
Ridge	Sequence of local maxima difficult to climb with single moves
Sensorless problem	Problem with no sensing; solution must work from every possible state
State space	All states reachable from the initial state via actions
Strong AI	View that machines can genuinely have minds
Transition model	RESULT(s, a): the state produced by doing a in s
Turing test	Behavioural test of machine intelligence via typed conversation
Uniform-cost search	Expands the node with lowest path cost g(n)
Weak AI	View that machines can act as if intelligent